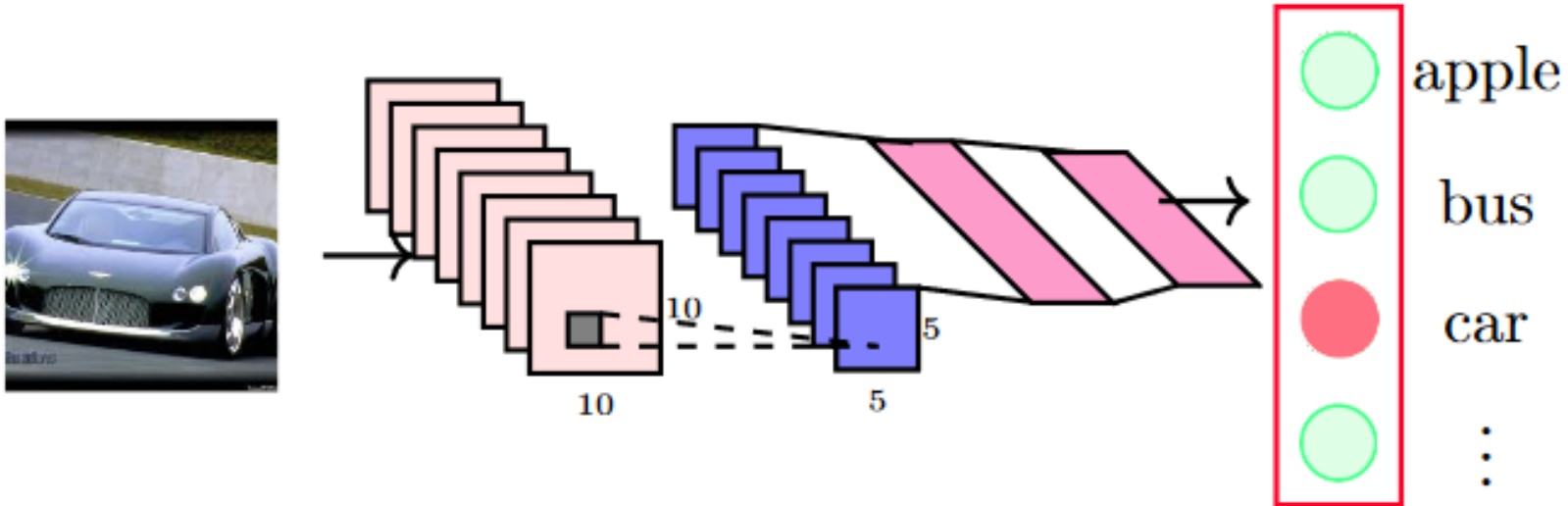


Sequence modeling – recurrent networks

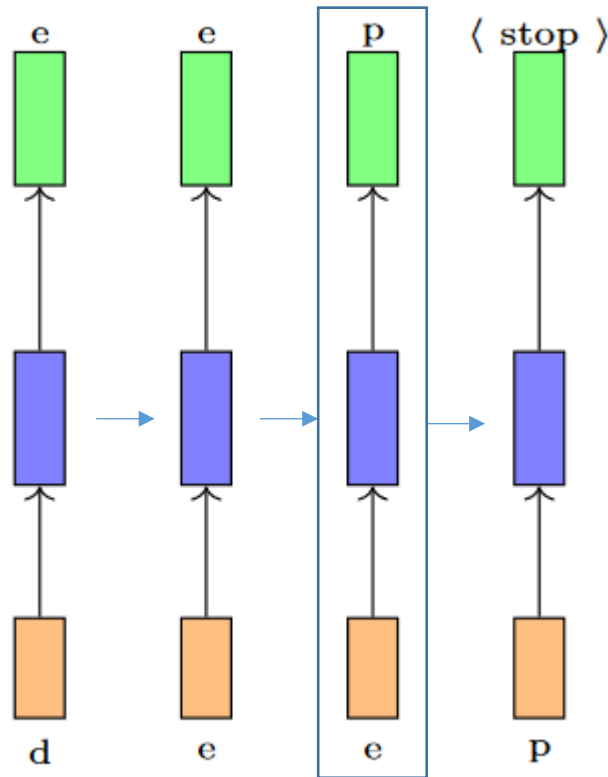
Biplab Banerjee

Sequential learning problem



- ✓ Each input is of fixed size
- ✓ Inputs are independent

Sequential learning problem

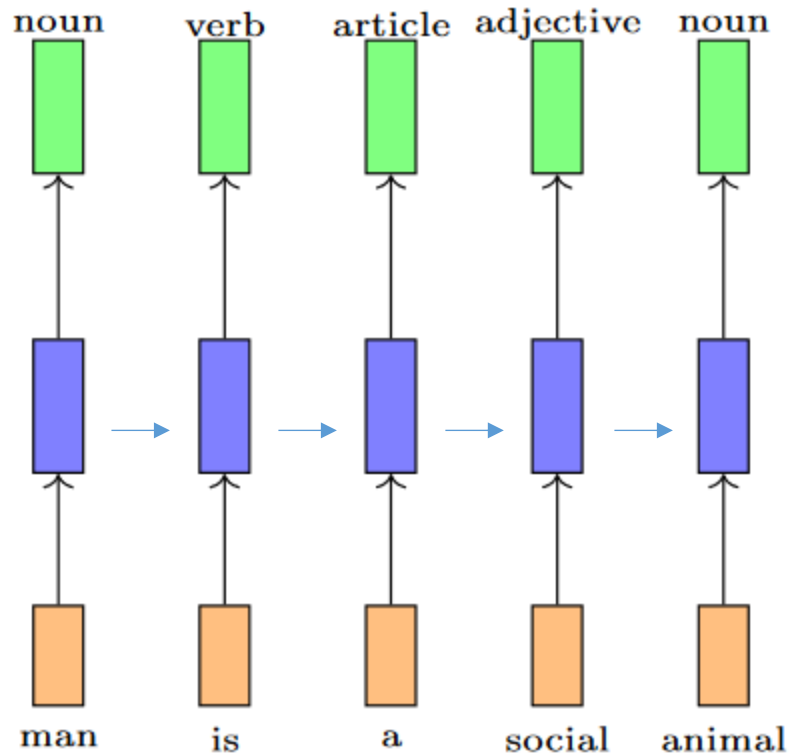


- ✓ Input lengths can vary
- ✓ Inputs are no longer independent
- ✓ At each time stamp, the same operation is Carried out

Given previous **information**
And current input, **predict**
Next input

Auto-correct application in e-mail

Sequential learning problem



Parts of speech tagging

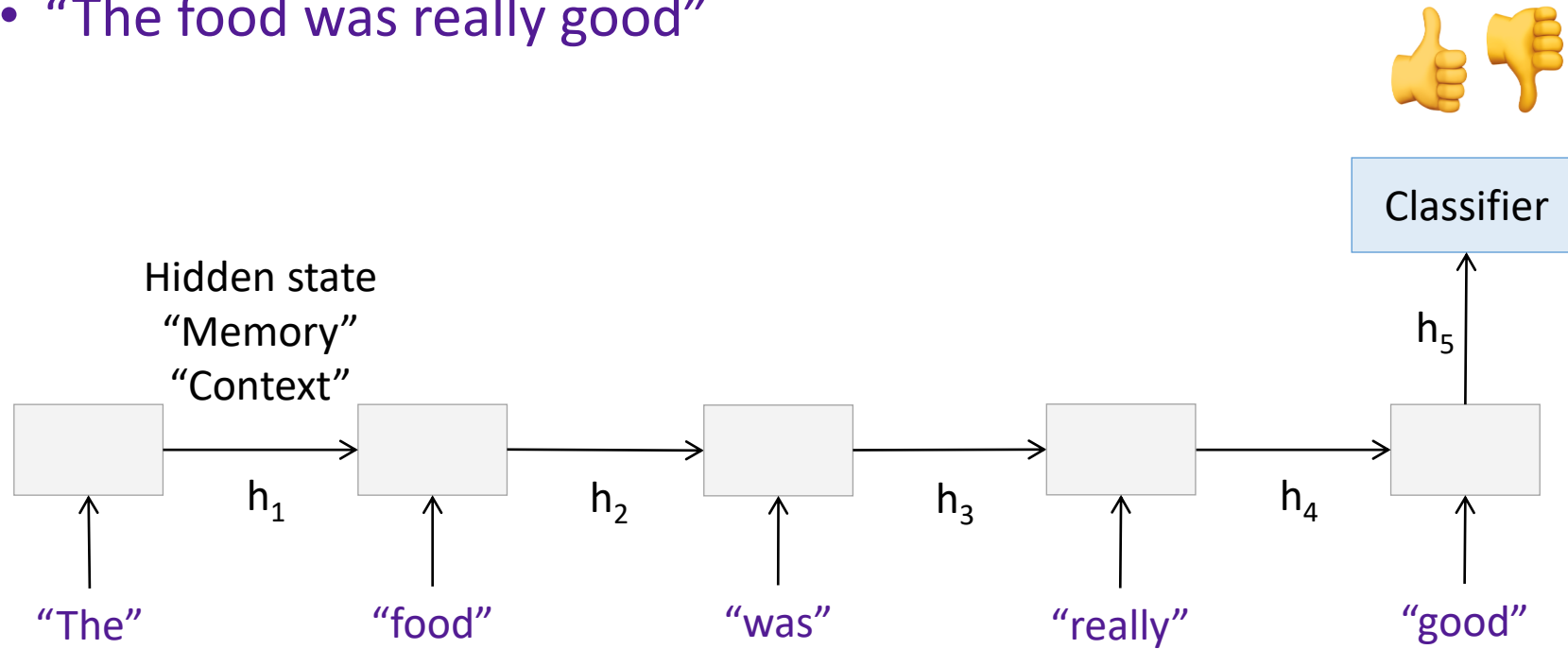
A set of previous time-stamps provide useful information
About the next time-stamp

Text classification

- **Sentiment classification:** classify a restaurant or movie or product review as positive or negative
 - “The food was really good”
 - “The vacuum cleaner broke within two weeks”
 - “The movie had slow parts, but overall was worth watching”
- What feature representation or predictor structure can we use for this problem?

Sentiment classification

- “The food was really good”



Recurrent Neural Network (RNN)

Machine translation



TranslateTurn off instant translation

EnglishSpanishFrenchDetect language

EnglishSpanishArabic

Translate

Correspondances

La Nature est un temple où de vivants piliers
Laissent parfois sortir de confuses paroles;
L'homme y passe à travers des forêts de symboles
Qui l'observent avec des regards familiers.
Comme de longs échos qui de loin se confondent
Dans une ténébreuse et profonde unité,
Vaste comme la nuit et comme la clarté,
Les parfums, les couleurs et les sons se répondent.
Il est des parfums frais comme des chairs d'enfants,
Doux comme les hautbois, verts comme les prairies,
— Et d'autres, corrompus, riches et triomphants,
Ayant l'expansion des choses infinies,
Comme l'ambre, le musc, le benjoin et l'encens,
Qui chantent les transports de l'esprit et des sens.
— Charles Baudelaire



693/5000

Matches

Nature is a temple where living pillars
Sometimes let out confused words;
Man goes through symbol forests
Which observe him with familiar eyes.
Like long echoes that by far merge
In a dark and deep unity,
As vast as the night and as clarity,
The perfumes, the colors and the sounds answer each
other.
There are fresh perfumes like children's flesh,
Sweet like oboes, green like meadows,
- And others, corrupt, rich and triumphant,
Having the expansion of infinite things,
Like amber, musk, benzoin and incense,
Who sing the transports of the mind and the senses.
- Charles Baudelaire

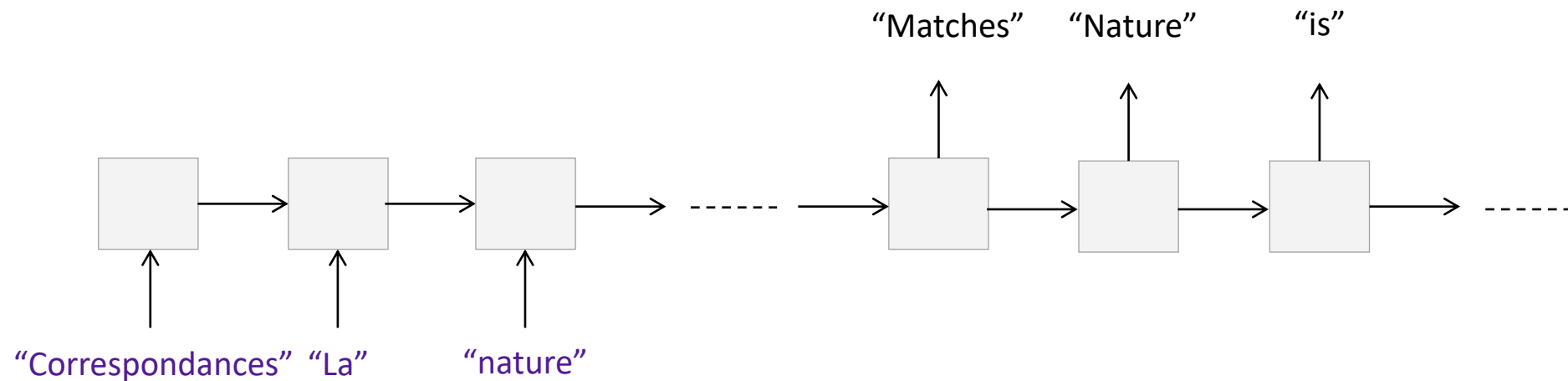


<https://translate.google.com/>

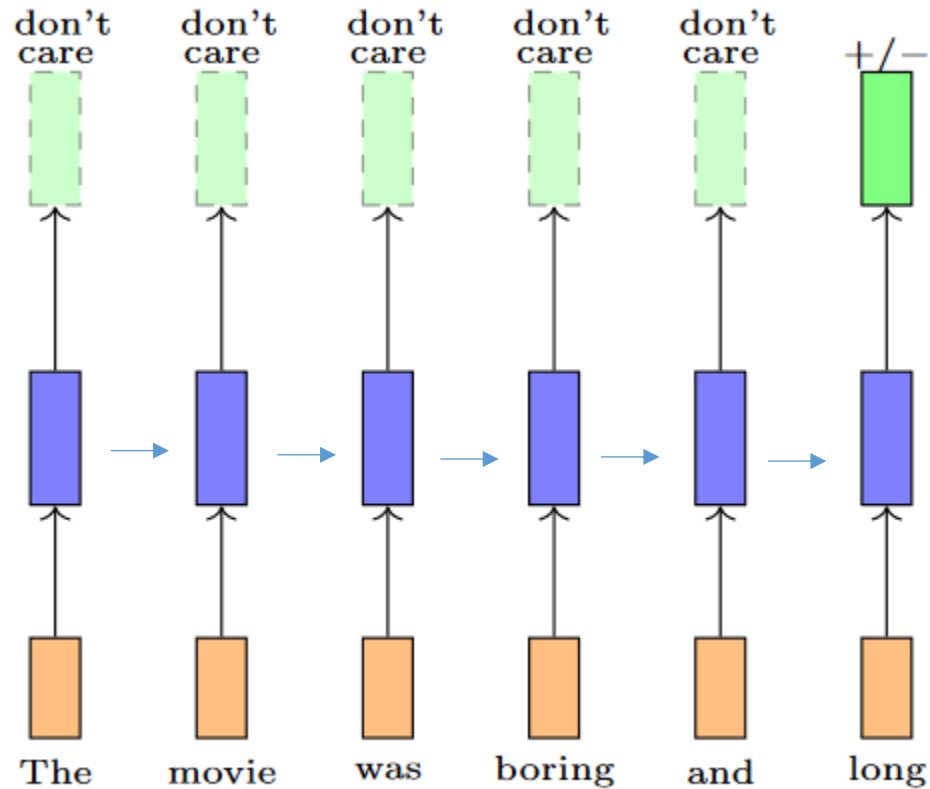
This file is meant for personal use by mteprad@nvidia.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Machine translation

- Multiple input – multiple output (or sequence to sequence)

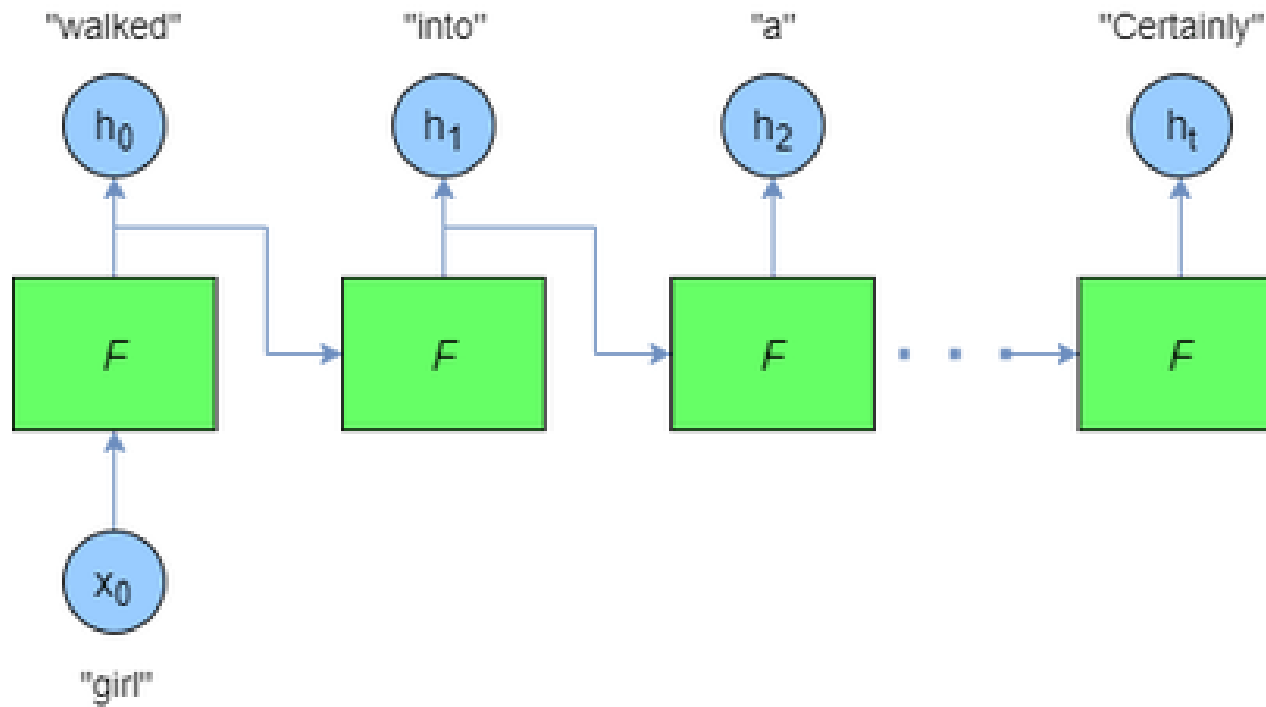


Different input output combinations



Movie review – many to one

Different input output combinations



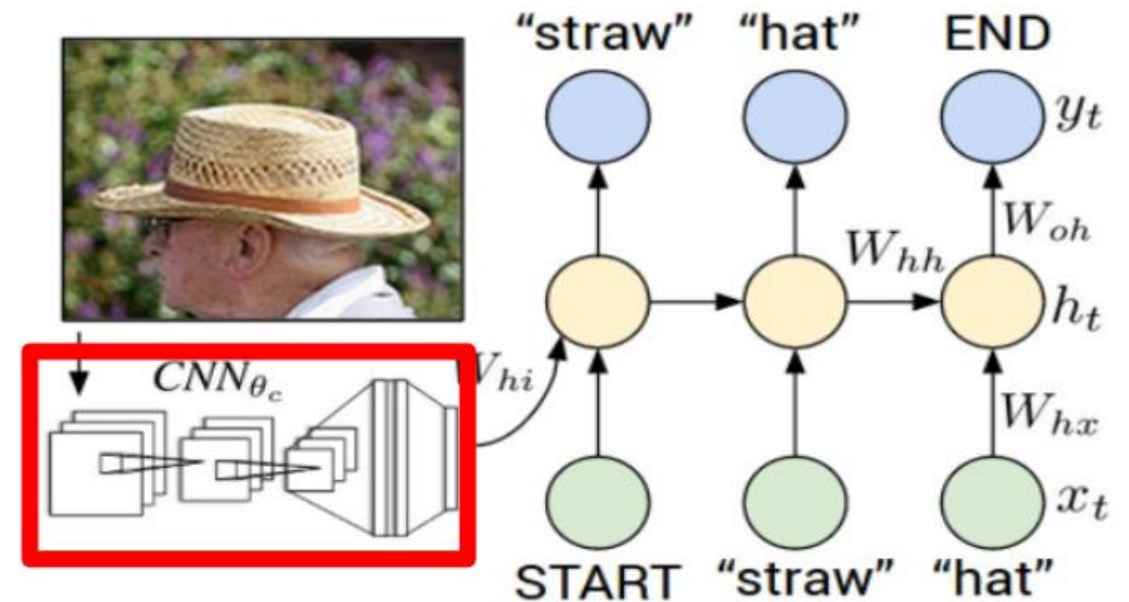
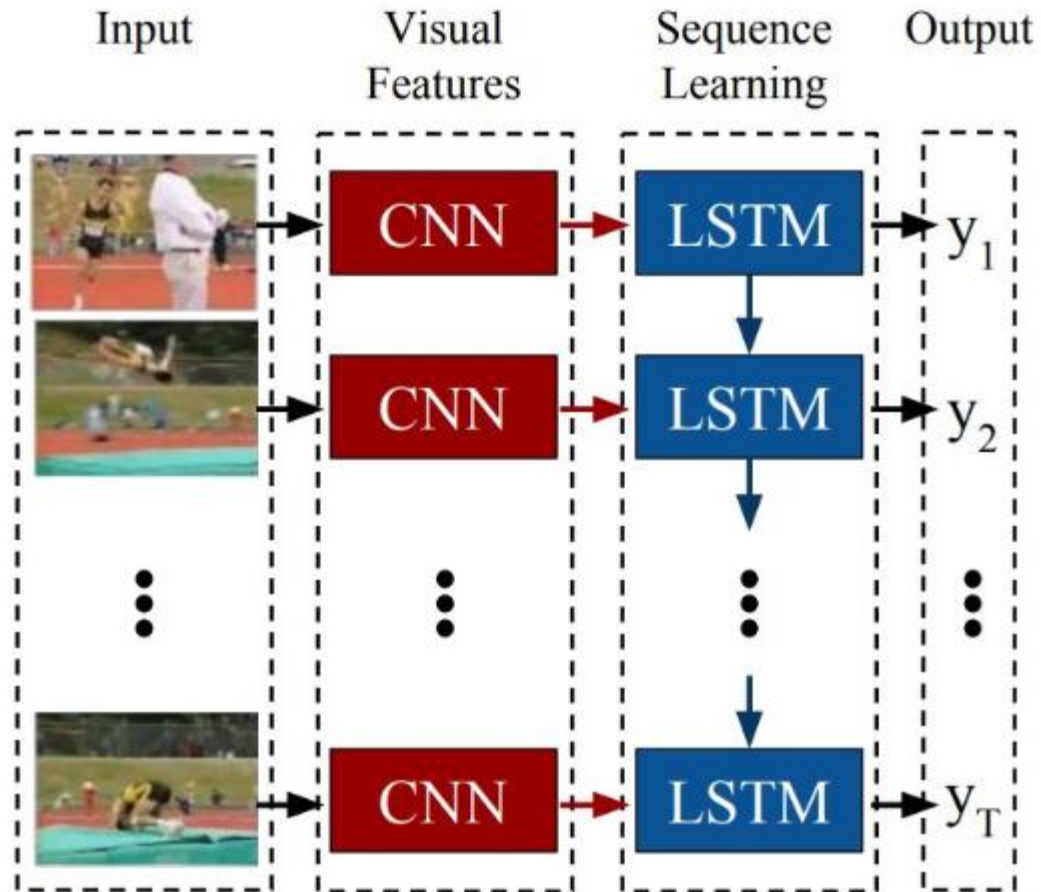
Many to Many -

Machine translation

Dialogue

Different input output combinations

Image and video captioning



Many to many and one to many

Single - Single



Feed-forward Network

Single - Multiple

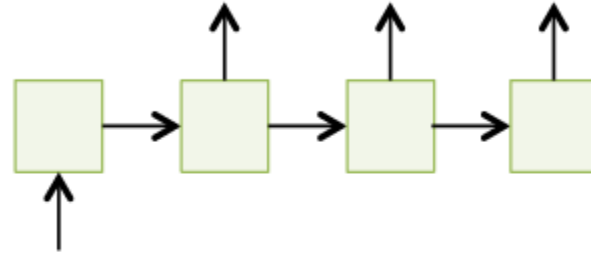
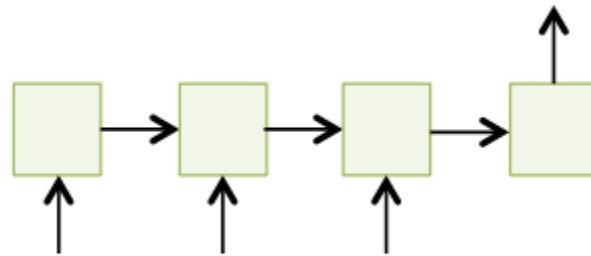


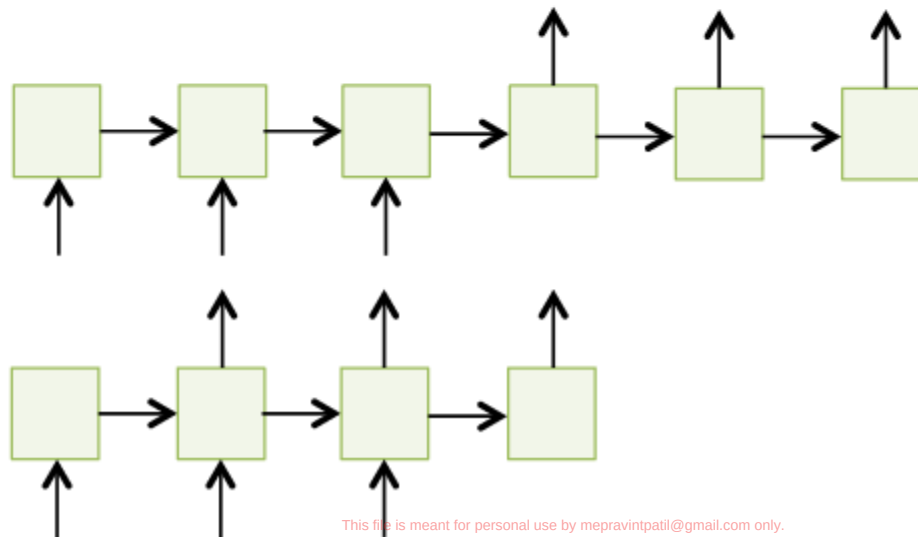
Image Captioning

Multiple - Single



Sentiment Classification

Multiple - Multiple



Translation

Image Captioning

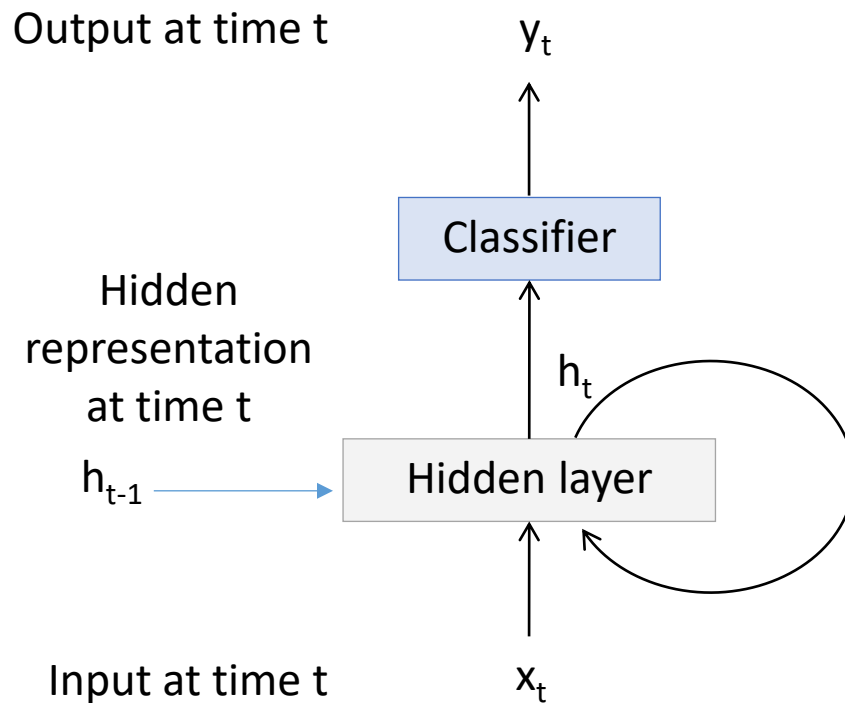
Sequential learning problem

$$P(w_1, \dots, w_m) = \prod_{i=1}^m P(w_i \mid w_1, \dots, w_{i-1}) \approx \prod_{i=1}^m P(w_i \mid w_{i-(n-1)}, \dots, w_{i-1})$$

The predicted word at a certain point can be inferred given a few previous words

Markovian Property of languages

Recurrent Neural Network (RNN)



Recurrence:

$$h_t = f_W(x_t, h_{t-1})$$

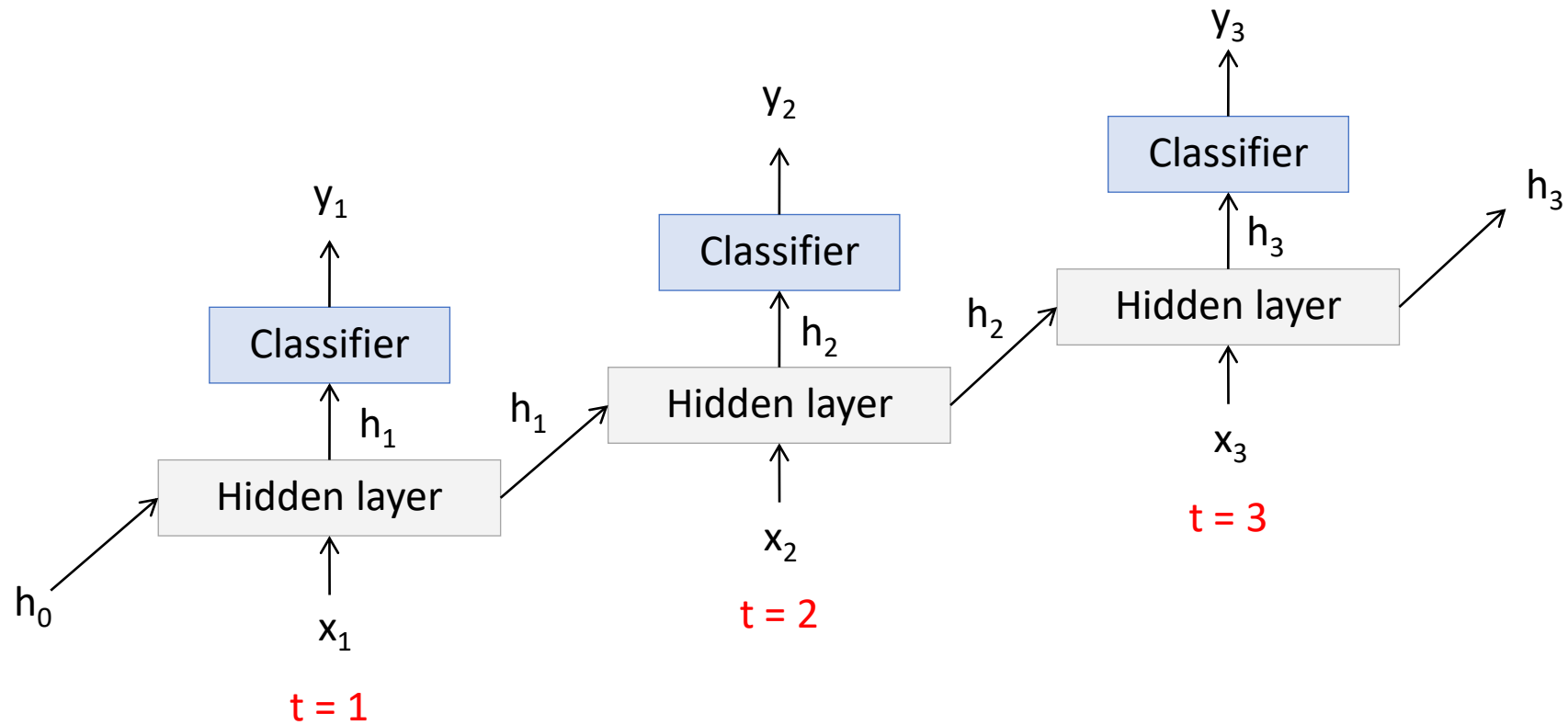
new
state

function
of W

input at
time t

old
state

Unrolling the RNN



Formulation

$$h(t) = \sigma (W h(t-1) + U x(t))$$

$$\hat{y}(t) = \text{softmax}(V h(t))$$

$$\hat{P}(x_{t+1} = v_j \mid x_t, \dots, x_1) = \hat{y}_{t,j}$$

Encodes the long-term dependency

$$J^{(t)}(\theta) = - \sum_{j=1}^{|V|} y_{t,j} \log \hat{y}_{t,j}$$

The loss function to be minimized

- ✓ Model for Many to Many prediction
- ✓ One output per time stamp
- ✓ $|V|$ denotes the number of words
- ✓ Training with softmax cross-entropy
- ✓ **The weights are shared for all the time-stamps**

Training is **Back-Propagation Through Time**

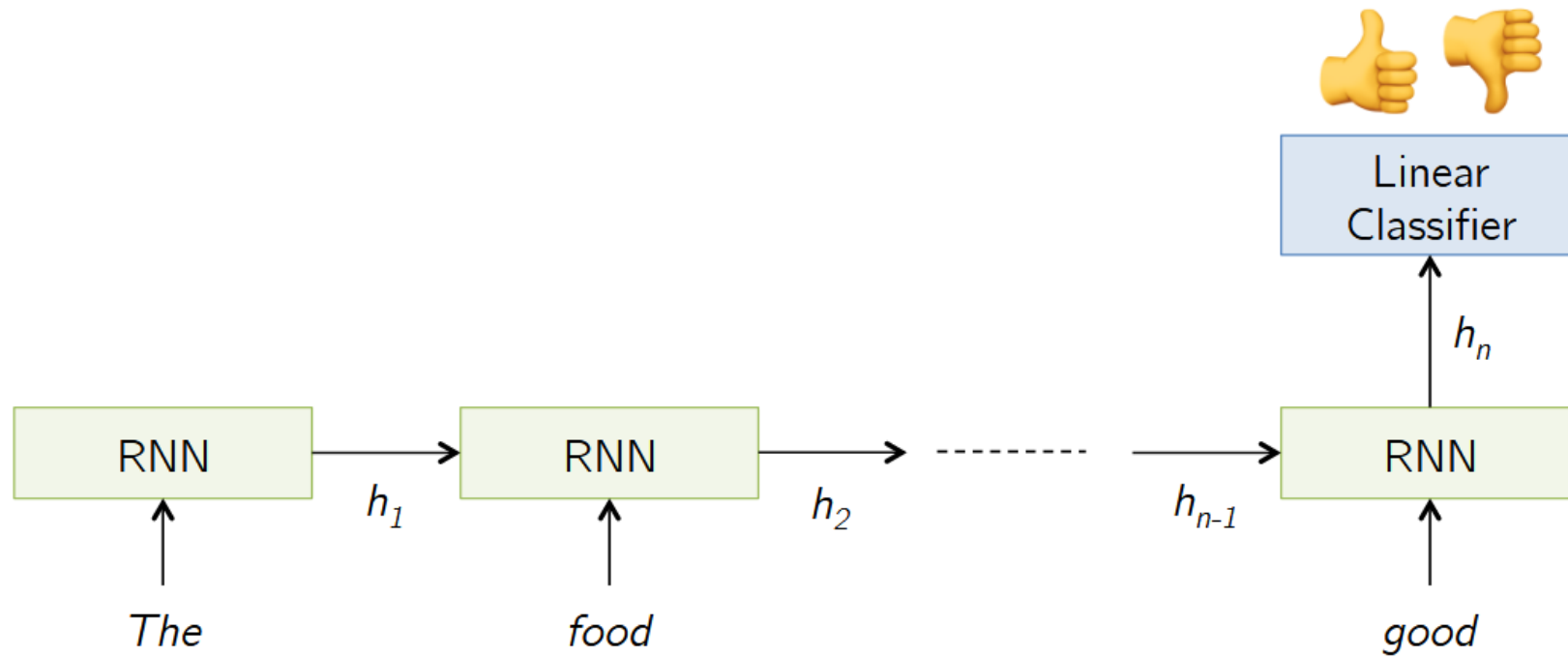
How to define the classifier

- There could be multiple possibilities for defining the classifier:
 - Ensemble of classifiers for all the states?
 - Single classifier at the final state?
- It is application dependent. Let us see two examples for sentiment classification and captioning

Example – sentiment classification

- Classify a restaurant review from Yelp! OR movie review from IMDB OR ... as positive or negative
- **Inputs:** Multiple words, one or more sentences
- **Outputs:** Positive / Negative classification
- “The food was really good”
- “The chicken crossed the road because it was uncooked”

Model-1



Model-2

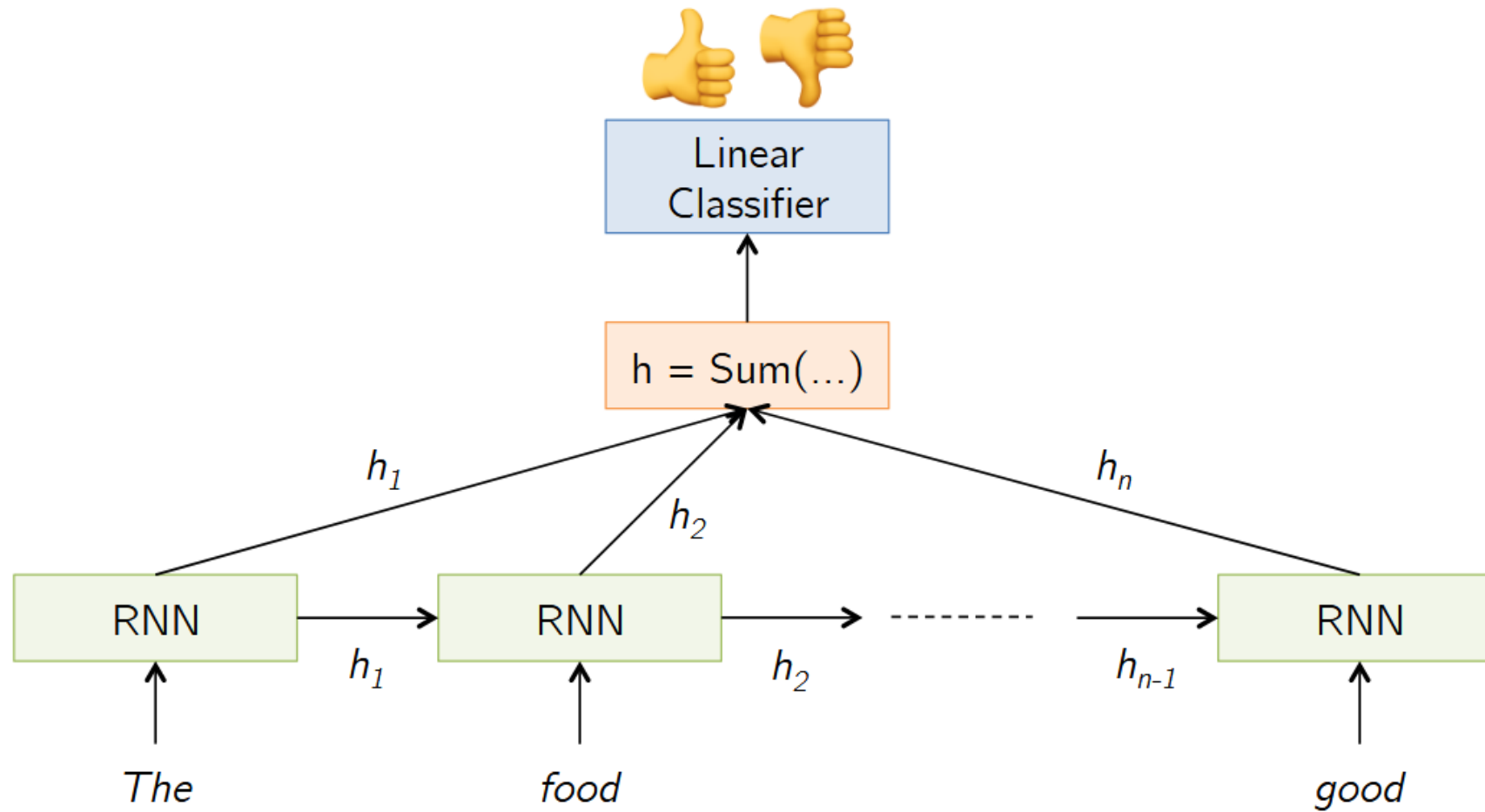
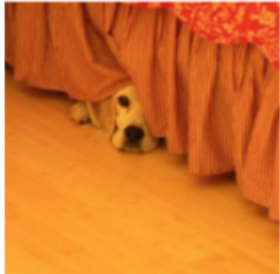


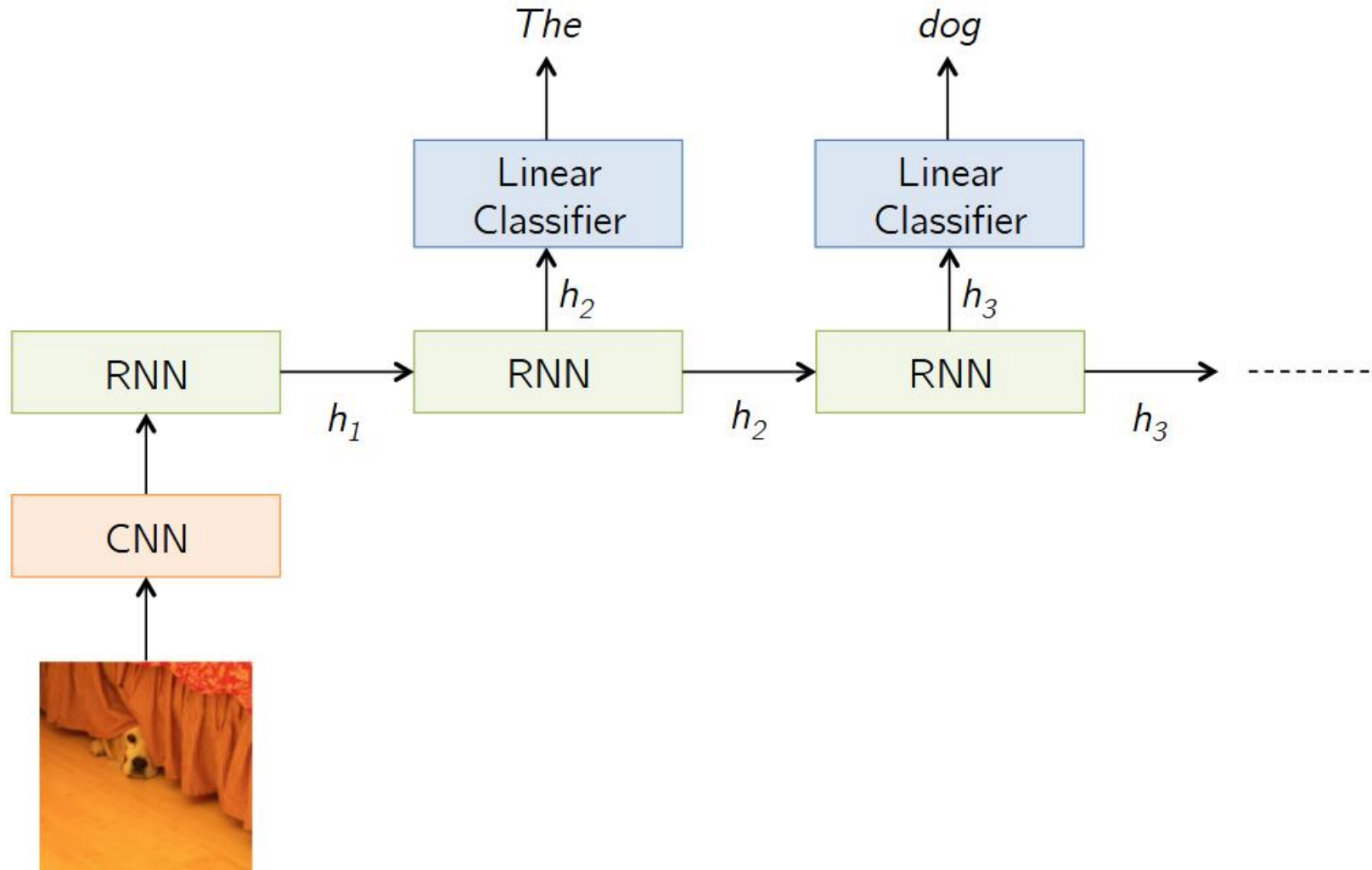
Image captioning

- Given an image, produce a sentence describing its contents
- **Inputs:** Image feature (from a CNN)
- **Outputs:** Multiple words (let's consider one sentence)



: The dog is hiding

Model



A person riding a motorcycle on a dirt road.



Two dogs play in the grass.



A herd of elephants walking across a dry grass field.



A group of young people playing a game of frisbee.



Two hockey players are fighting over the puck.



A close up of a cat laying on a couch.



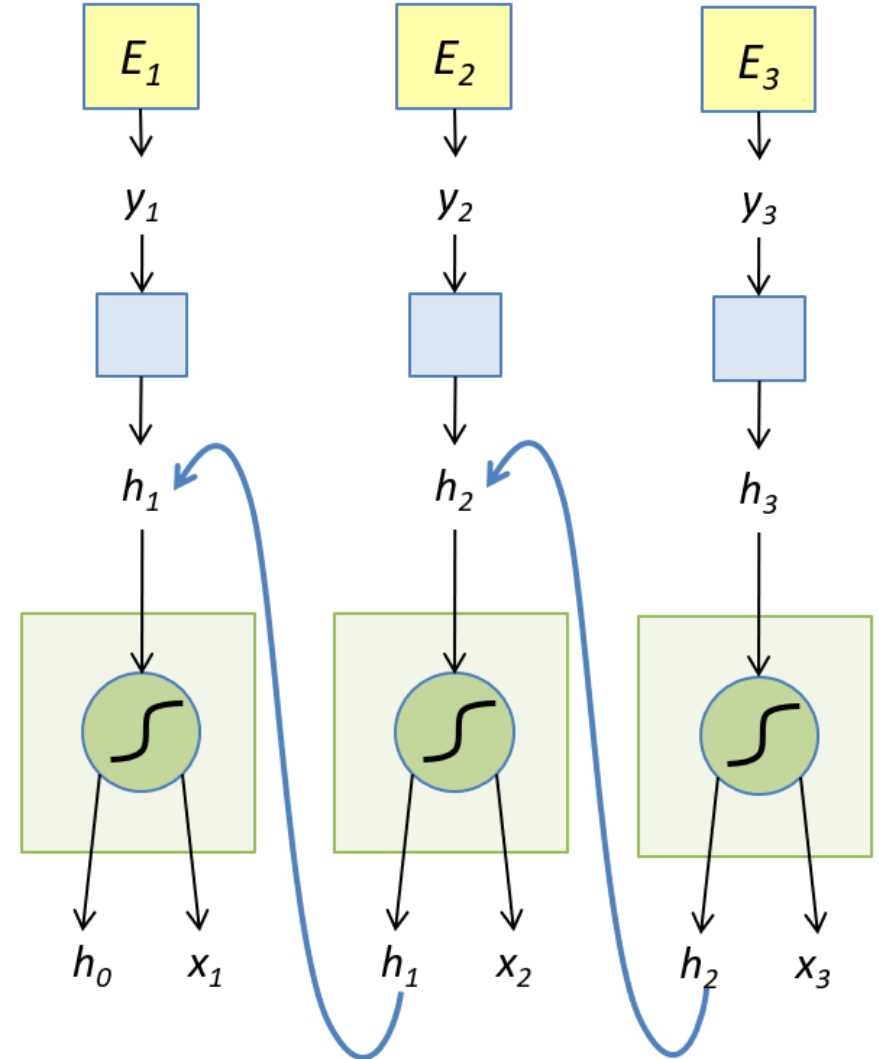
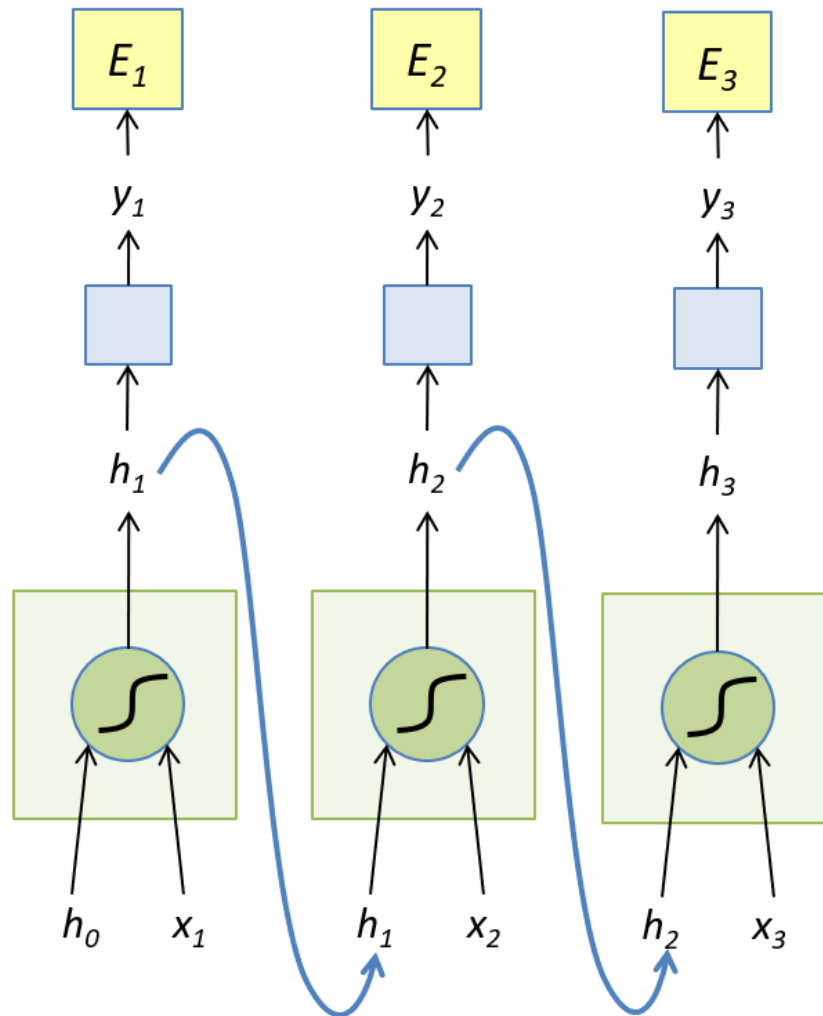
Some highlights

- RNN takes the previous output or hidden state as inputs at every time together with the usual input. This means that the composite input at time t has some historical information about the happenings at time $T < t$.
- RNNs gained popularity in sequence modeling without forgetting important past information as their intermediate values (state) can store information about the past inputs for a time that is not fixed a priori.

Back propagation through time

- Considered to be the standard method used to train RNNs.
- The unfolded network (used during forward pass) is treated as one feed-forward network that accepts the whole time series as input.
- The weight updates are computed for each copy in the unfolded network, then accumulated and applied to the model weights and biases.

Forward and backward pass



Calculating the gradients with U,V,W

A standard RNN is

$$a_t = Wh_{t-1} + Ux_t + b$$

$$h_t = \phi(a_t)$$

$$y_t = Vh_t + c$$

and the total loss is

$$E = \sum_{t=0}^T E_t$$

where each E_t depends on y_t .

Since

$$y_t = Vh_t + c$$

we have

$$\frac{\partial E}{\partial V} = \sum_{t=0}^T \frac{\partial E_t}{\partial V}$$

Using chain rule,

$$\frac{\partial E_t}{\partial V} = \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial V}$$

So the result is

$$\frac{\partial E}{\partial V} = \sum_{t=0}^T \delta_t^{(y)} h_t^T$$

where

$$\delta_t^{(y)} = \frac{\partial E_t}{\partial y_t}$$

This is the output-layer error.

Since

$$a_t = Wh_{t-1} + Ux_t + b$$

we get

$$\frac{\partial E}{\partial U} = \sum_{t=0}^T \frac{\partial E}{\partial a_t} \frac{\partial a_t}{\partial U}$$

Hence

$$\frac{\partial E}{\partial U} = \sum_{t=0}^T \delta_t x_t^T$$

$$\frac{\partial E}{\partial h_t} = \frac{\partial E_t}{\partial h_t} + \frac{\partial E}{\partial a_{t+1}} \frac{\partial a_{t+1}}{\partial h_t}$$

By definition,

$$\delta_t = \frac{\partial E}{\partial a_t}$$

Since

$$h_t = \phi(a_t)$$

we use chain rule:

$$\frac{\partial E}{\partial a_t} = \frac{\partial E}{\partial h_t} \frac{\partial h_t}{\partial a_t}$$

Because ϕ acts elementwise,

$$\frac{\partial h_t}{\partial a_t} = \phi'(a_t)$$

so

$$\delta_t = \frac{\partial E}{\partial h_t} \odot \phi'(a_t)$$

Current-loss contribution

Since

$$y_t = Vh_t + c$$

we get

$$\frac{\partial E_t}{\partial h_t} = V^T \frac{\partial E_t}{\partial y_t}$$

Define output error

$$\delta_t^{(y)} := \frac{\partial E_t}{\partial y_t}$$

Then

$$\frac{\partial E_t}{\partial h_t} = V^T \delta_t^{(y)}$$

Future contribution

Since

$$a_{t+1} = Wh_t + Ux_{t+1} + b$$

we have

$$\frac{\partial a_{t+1}}{\partial h_t} = W$$

So

$$\frac{\partial E}{\partial a_{t+1}} \frac{\partial a_{t+1}}{\partial h_t} = W^T \delta_{t+1}$$

Therefore

$$\frac{\partial E}{\partial h_t} = V^T \delta_t^{(y)} + W^T \delta_{t+1}$$

Substitute this back into the earlier formula for δ_t :

$$\delta_t = \left(V^T \delta_t^{(y)} + W^T \delta_{t+1} \right) \odot \phi'(a_t)$$

Recall

$$a_t = Wh_{t-1} + Ux_t + b$$

Only the term Wh_{t-1} depends on W .

So differentiating with respect to W gives an outer product with h_{t-1} . Therefore the contribution at time t is

$$\delta_t h_{t-1}^T$$

Hence the compact formula is

$$\frac{\partial E}{\partial W} = \sum_{t=0}^T \delta_t h_{t-1}^T$$

$$\frac{\partial E}{\partial W} = \sum_{t=0}^T \left[\left(V^T \delta_t^{(y)} + W^T \delta_{t+1} \right) \odot \phi'(a_t) \right] h_{t-1}^T$$

For an RNN,

$$a_t = Wh_{t-1} + Ux_t + b, \quad h_t = \phi(a_t), \quad y_t = Vh_t + c$$

with total loss

$$E = \sum_{t=0}^T E_t$$

the gradient with respect to the recurrent weight W is

$$\frac{\partial E}{\partial W} = \sum_{t=0}^T \delta_t h_{t-1}^T$$

where the hidden error term δ_t is computed by backpropagation through time as

$$\delta_t = \left(V^T \delta_t^{(y)} + W^T \delta_{t+1} \right) \odot \phi'(a_t), \quad \delta_t^{(y)} = \frac{\partial E_t}{\partial y_t}$$

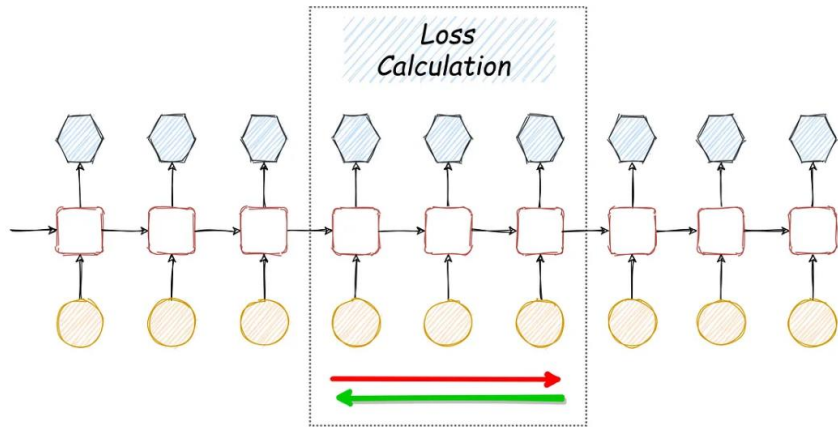
The exploding gradient problem arises because δ_t depends on δ_{t+1} , which depends on δ_{t+2} , and so on.

Therefore, when expanded backward through many time steps, the gradient contains repeated products of terms involving W^T , such as

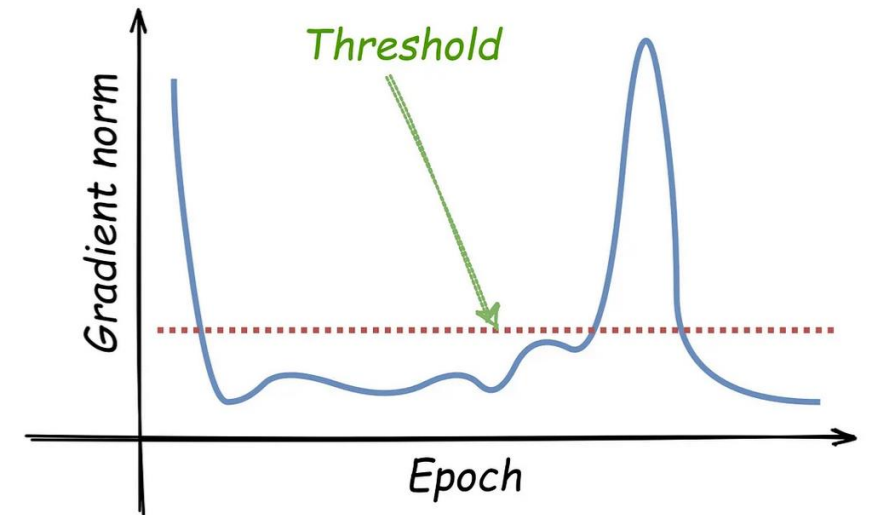
$$W^T W^T W^T \dots$$

along with activation derivatives. If the norm of these factors is greater than 1, the product grows exponentially with the number of time steps. As a result, δ_t becomes very large, and hence $\frac{\partial E}{\partial W}$ also becomes very large.

Solutions (locality and clipping)

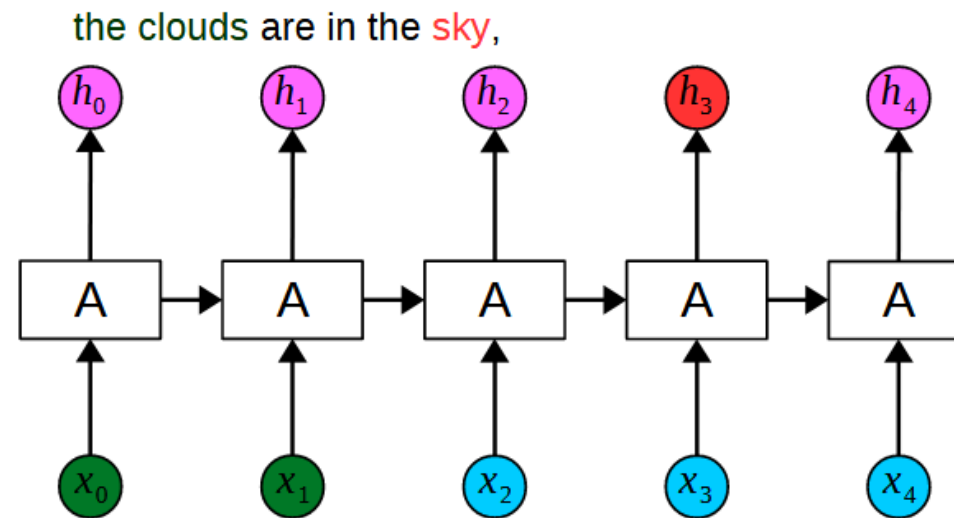


Instead of processing the entire sequence, process the forward/backward passes in chunks



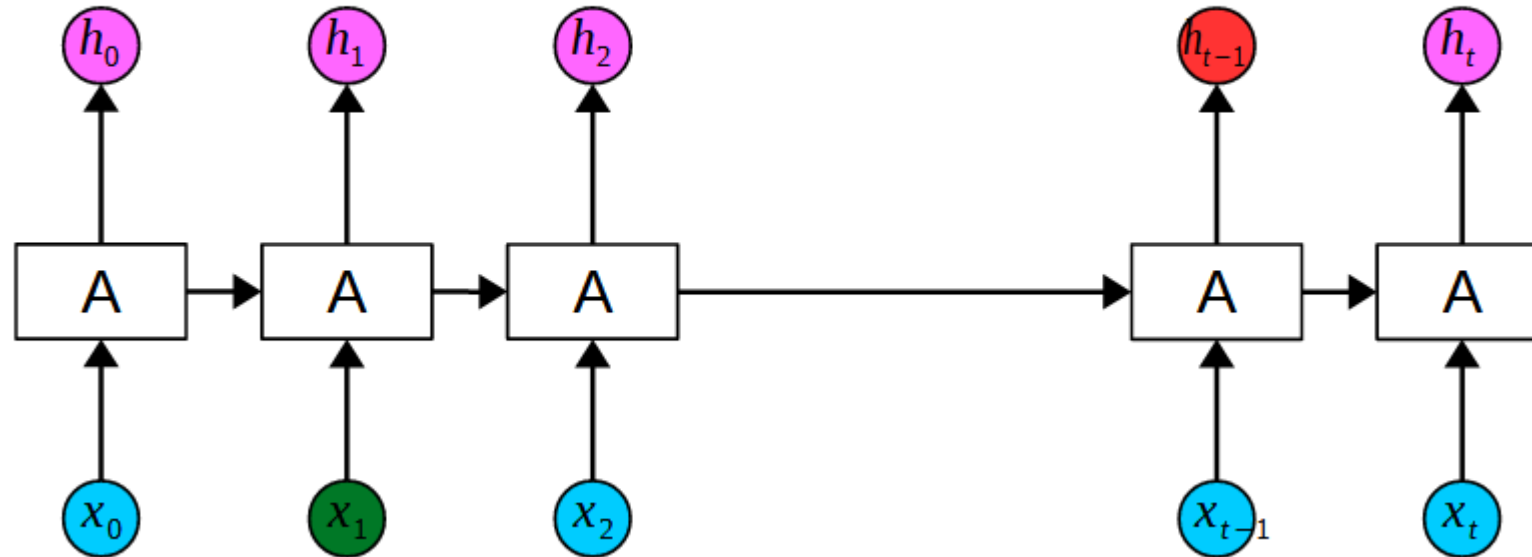
Short term dependency

Language model trying to predict the next word based on the previous ones



Long term dependency

I grew up in India... I speak fluent Hindi.



RNN fails to preserve the long term dependency due to the vanishing gradient problem

We should distinguish between what to remember and what to forget more wisely

LSTM (Long short term memory) to the rescue

Influence of older state on newer state

$$h_t = \phi(W h_{t-1} + U x_t + b)$$

By chain rule,

$$\frac{\partial h_t}{\partial h_k} = \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \cdots \frac{\partial h_{k+1}}{\partial h_k}$$

For one step,

$$\frac{\partial h_i}{\partial h_{i-1}} = D_i W$$

where

$$D_i = \text{diag}(\phi'(a_i))$$

So,

$$\frac{\partial h_t}{\partial h_k} = (D_t W)(D_{t-1} W) \cdots (D_{k+1} W)$$

Take norms on both sides:

$$\left\| \frac{\partial h_t}{\partial h_k} \right\| \leq \prod_{i=k+1}^t \|D_i\| \|W\|$$

If the activation is sigmoid or tanh, then usually

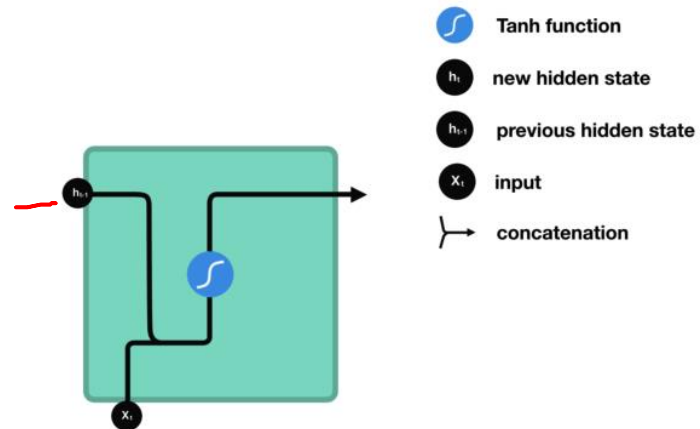
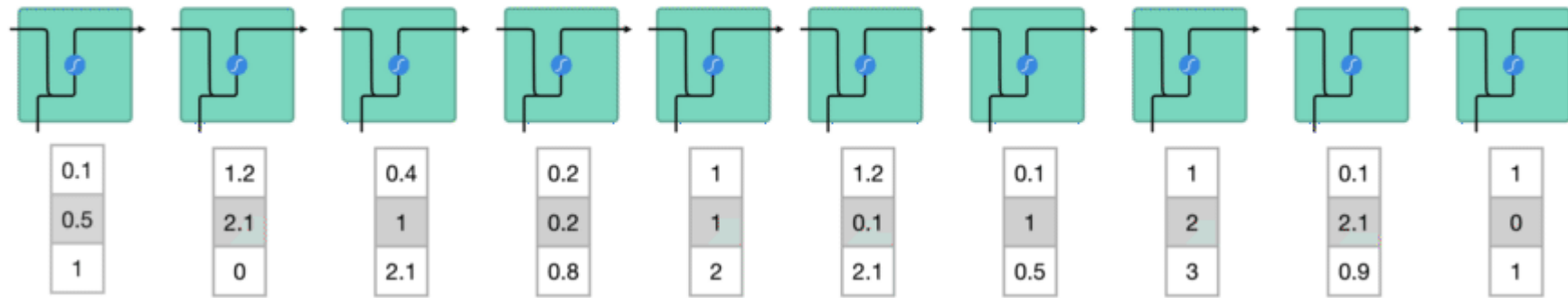
$$|\phi'(a_i)| < 1$$

so typically

$$\|D_i\| < 1$$

Hence, if the product $\|D_i\| \|W\|$ is less than 1 on average, then

$$\left\| \frac{\partial h_t}{\partial h_k} \right\| \rightarrow 0 \quad \text{as } t - k \text{ increases}$$



How RNN works! – trying to process all the words and update memory on that basis

Some intuition

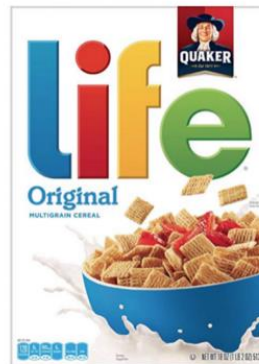
Customers Review 2,491



Thanos

September 2018
Verified Purchase

Amazing! This box of cereal gave me a perfectly balanced breakfast, as all things should be. I only ate half of it but will definitely be buying again!



A Box of Cereal
\$3.99

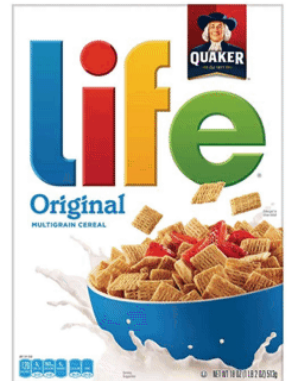
Customers Review 2,491



Thanos

September 2018
Verified Purchase

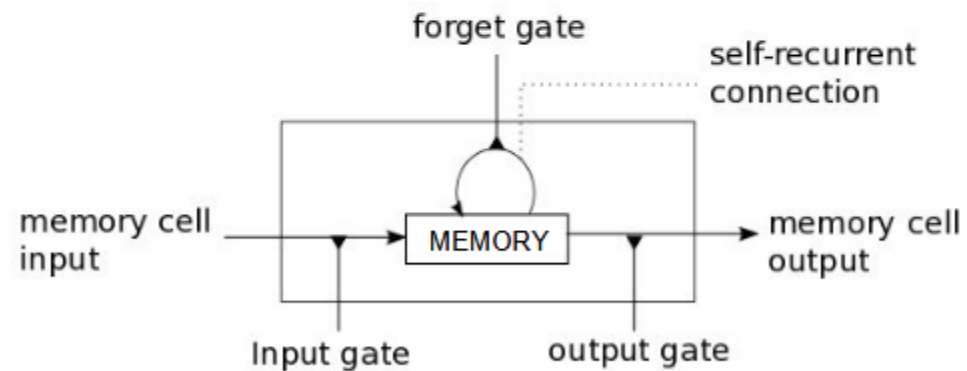
~~Amazing!~~ This box of cereal gave me a perfectly balanced breakfast, as all things should be. I only ate half of it but will definitely be buying again!



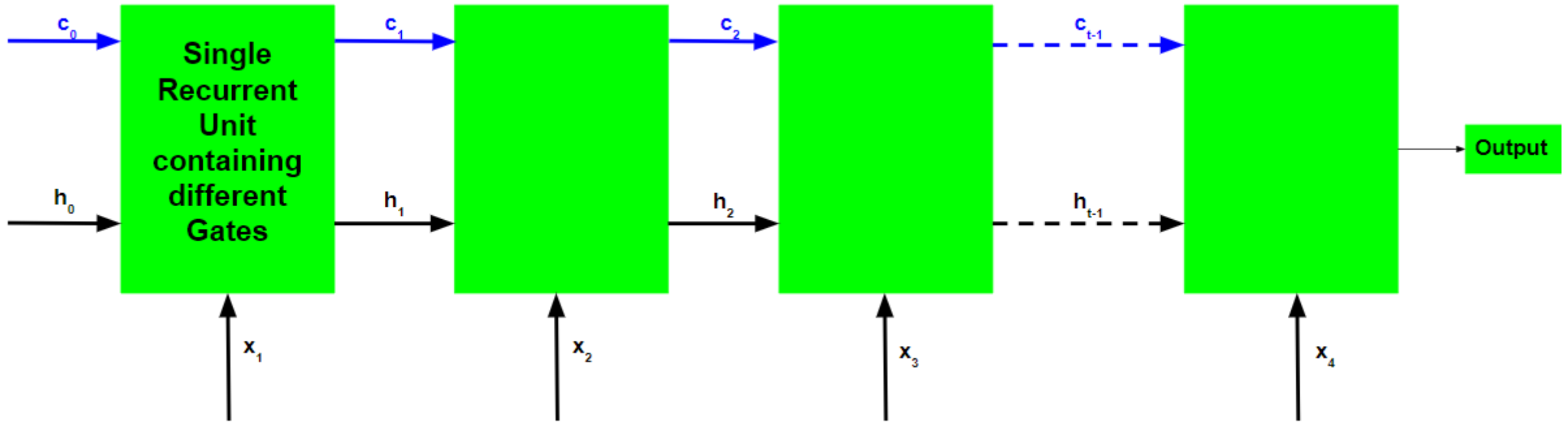
A Box of Cereal
\$3.99

LSTM

Central Idea: A **memory cell** (interchangeably **block**) which can maintain its state over time, consisting of an explicit memory (aka the **cell state vector**) and **gating units** which regulate the information flow into and out of the memory.



LSTM Memory Cell



If we had only one state, then the same vector would have to do both jobs:

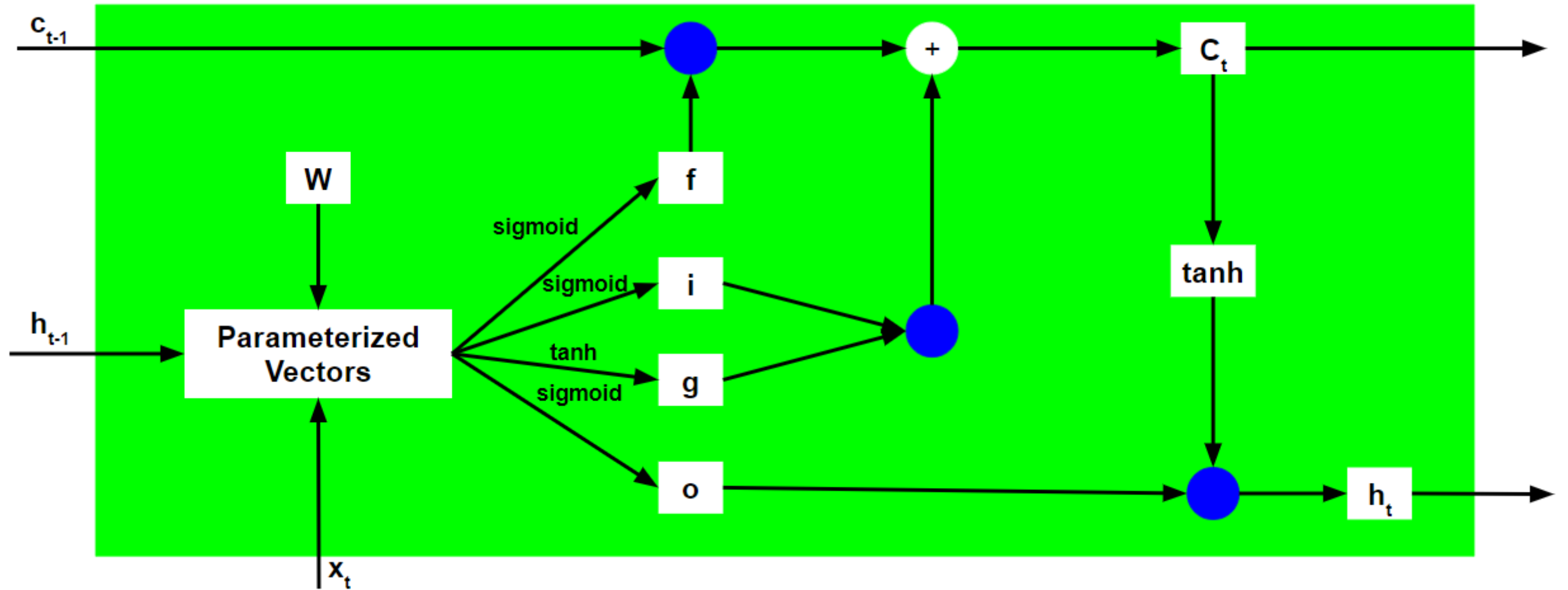
- preserve long-term memory
- provide immediate output

That is difficult.

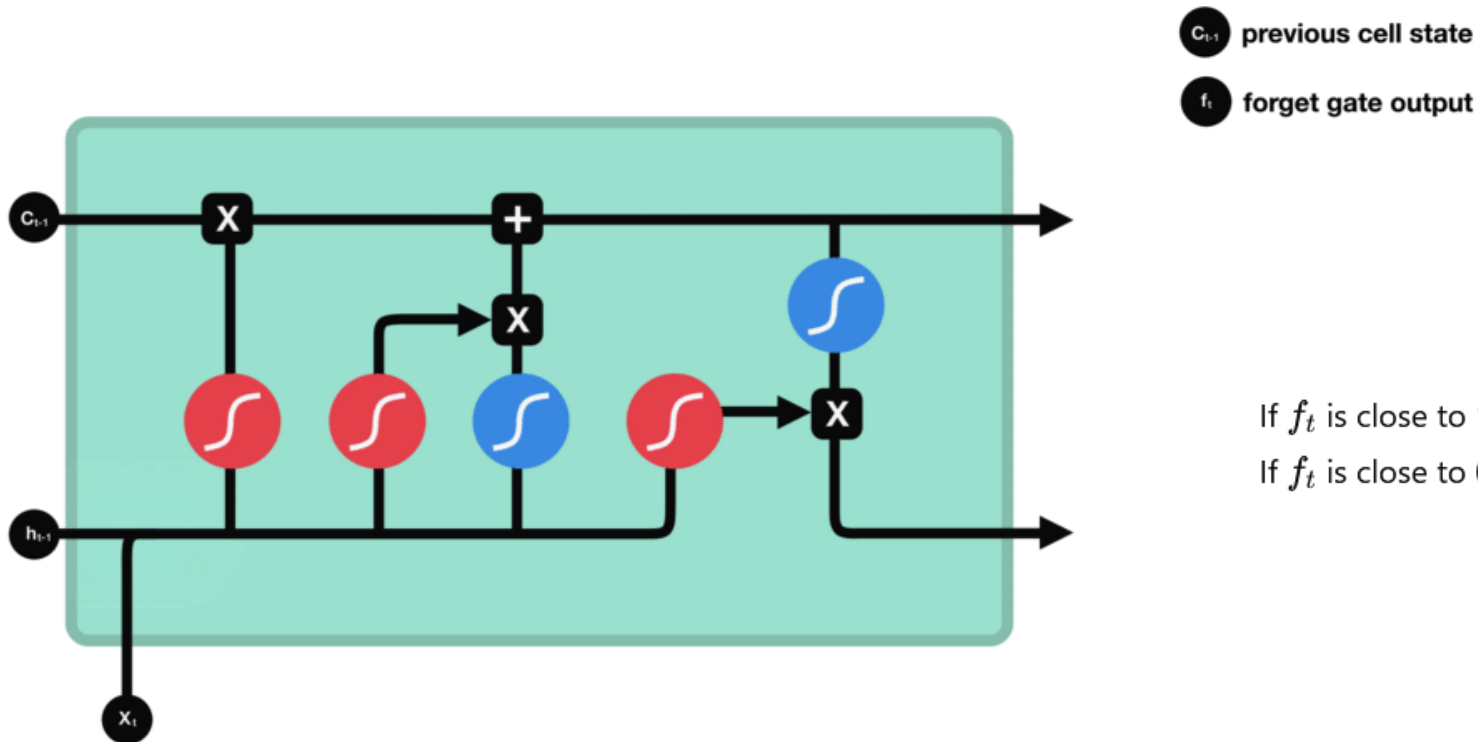
LSTM separates these roles:

- c_t : memory storage
- h_t : current usable output

This separation is one reason LSTM handles long dependencies better than vanilla RNN.



Forget gate



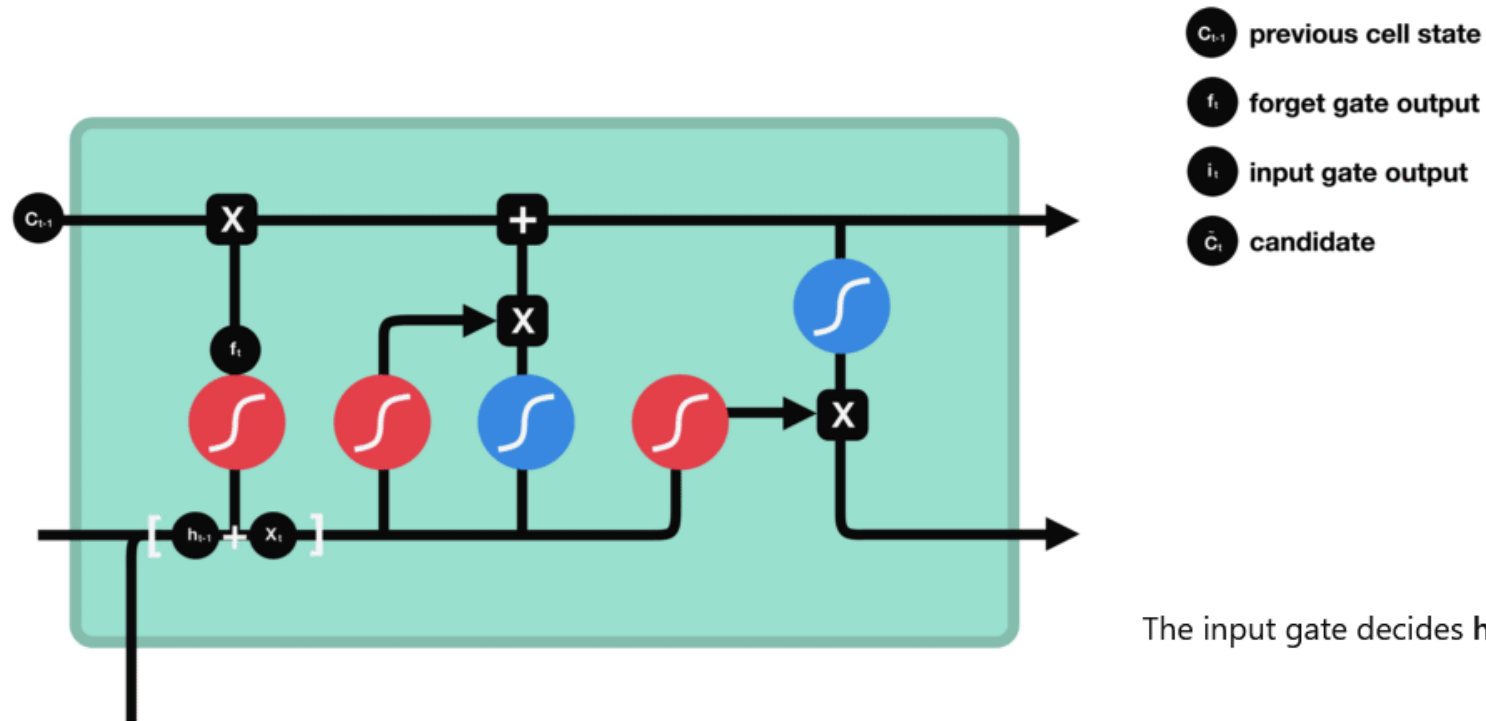
$$f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f)$$

If f_t is close to 1, old memory is retained.

If f_t is close to 0, old memory is discarded.

This gate decides what information should be thrown away or kept. Information from the previous hidden state and information from the current input is passed through the sigmoid function. Values come out between 0 and 1. The closer to 0 means to forget, and the closer to 1 means to keep.

Input gate



The input gate decides how much new information should be written into memory:

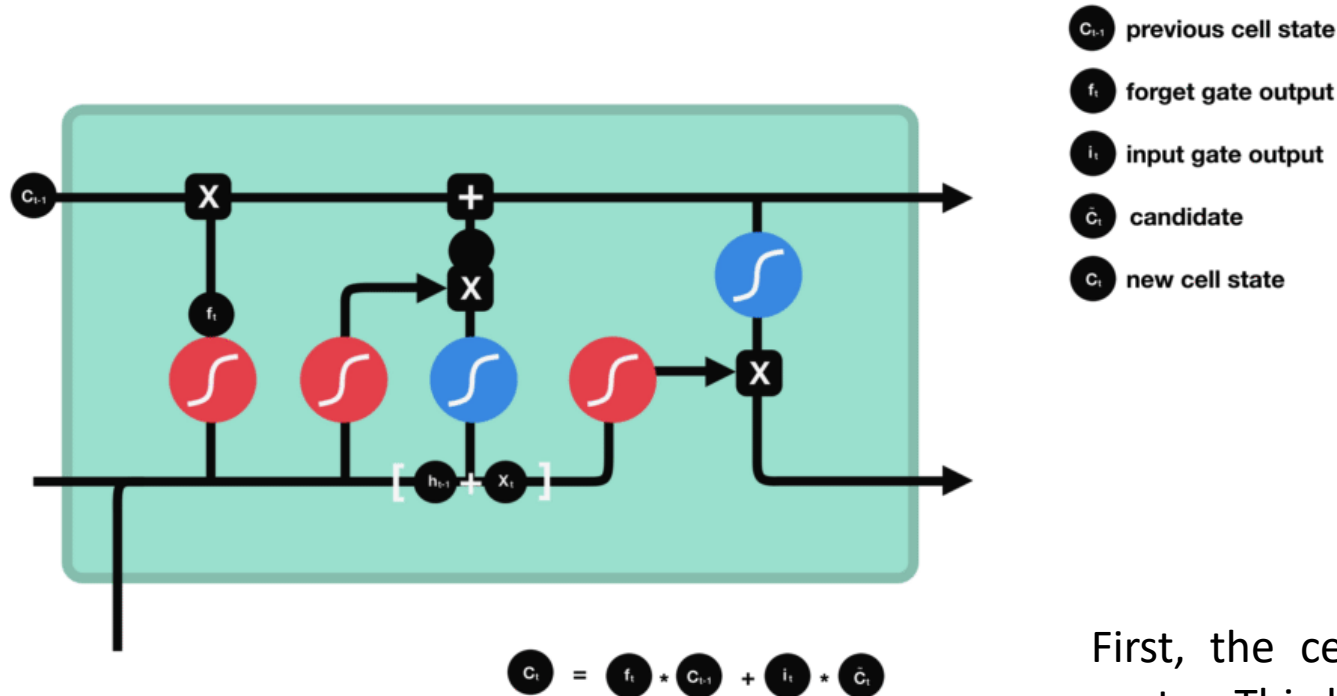
$$i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i)$$

A candidate memory is computed as

$$\tilde{c}_t = \tanh(W_c h_{t-1} + U_c x_t + b_c)$$

Then the input gate controls how much of this candidate enters the cell state.

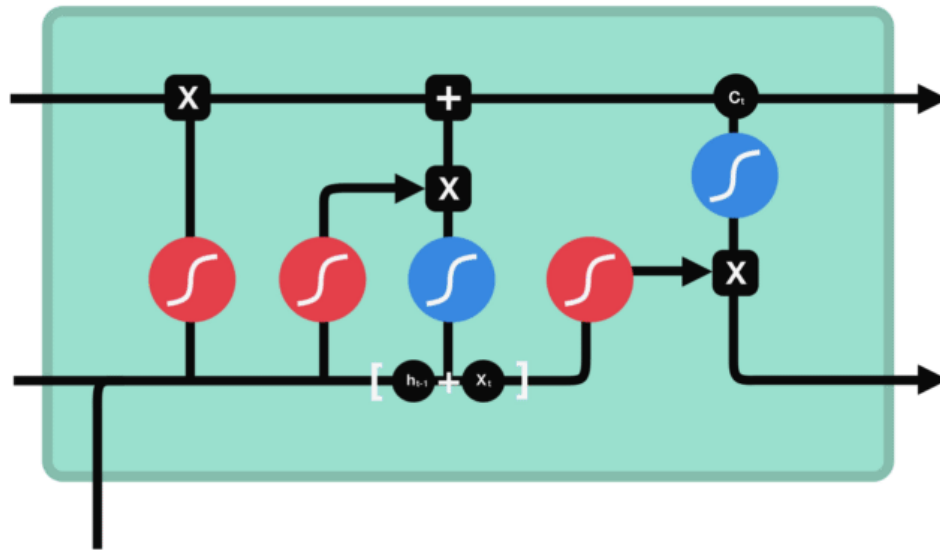
Updating the cell state



$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

First, the cell state gets pointwise multiplied by the forget vector. This has a possibility of dropping values in the cell state if it gets multiplied by values near 0. Then we take the output from the input gate and do a pointwise addition which updates the cell state to new values that the neural network finds relevant. That gives us our new cell state.

Output gate



- c_{t-1} previous cell state
- f_t forget gate output
- i_t input gate output
- $\tilde{c}_t$ candidate
- c_t new cell state
- o_t output gate output
- h_t hidden state

$$o_t = \sigma(W_o h_{t-1} + U_o x_t + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

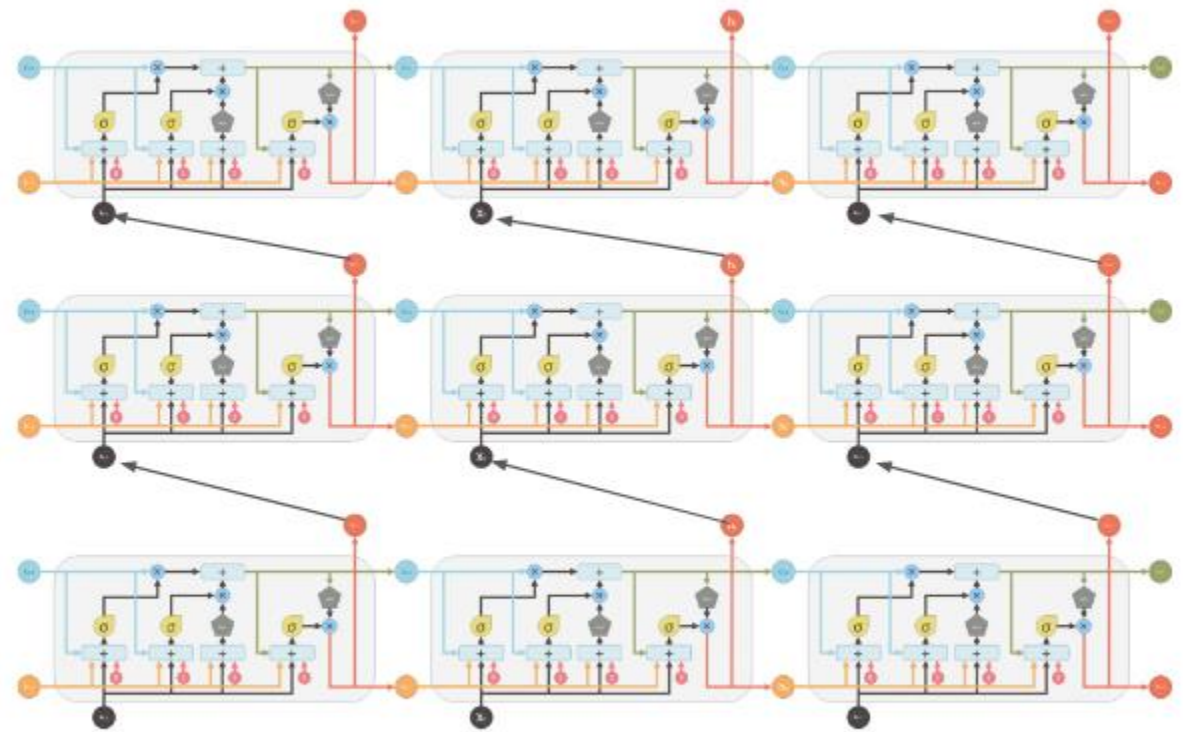
First, we pass the previous hidden state and the current input into a sigmoid function. Then we pass the newly modified cell state to the tanh function. We multiply the tanh output with the sigmoid output to decide what information the hidden state should carry. The output is the hidden state.

Training LSTM

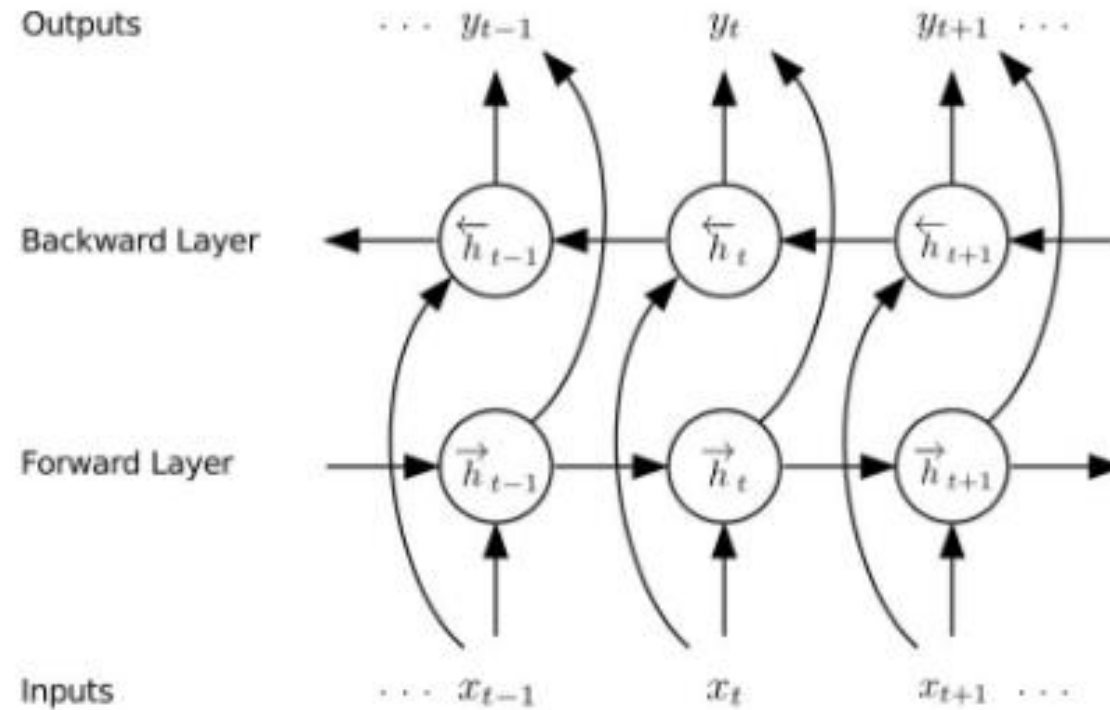
- Backpropagation through time
- Each cell has many parameters (W_f , W_i , W_C , W_o)
 - Generally requires lots of training data.
 - Requires lots of compute time that exploits GPU clusters.

Deep LSTM

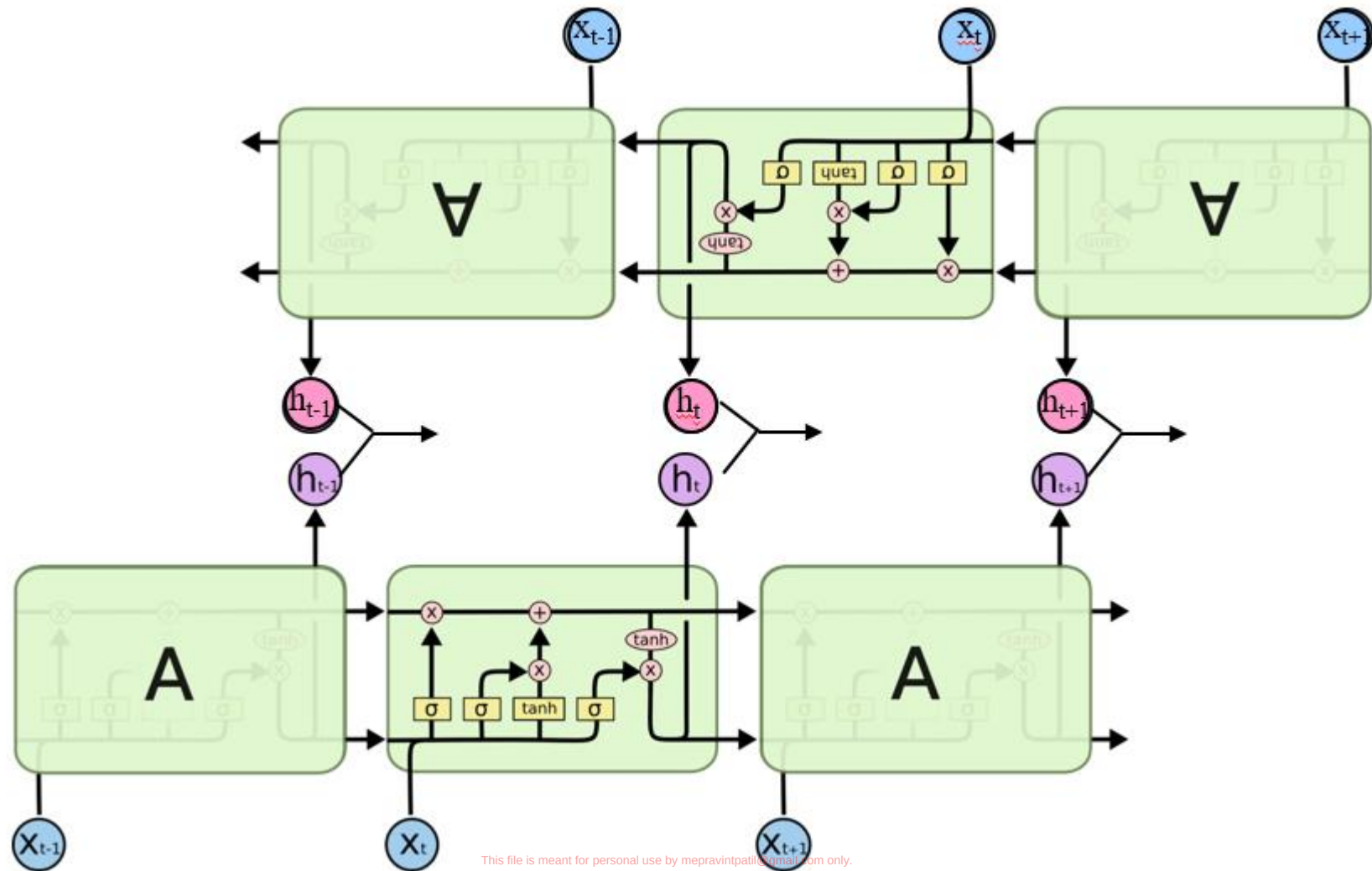
- Deep LSTMs can be created by stacking multiple LSTM layers vertically, with the output sequence of one layer forming the input sequence of the next (in addition to recurrent connections within the same layer)
- Increases the number of parameters - but given sufficient data, performs significantly better than single-layer LSTMs (Graves et al. 2013)
- Dropout usually applied only to non-recurrent edges, including between layers



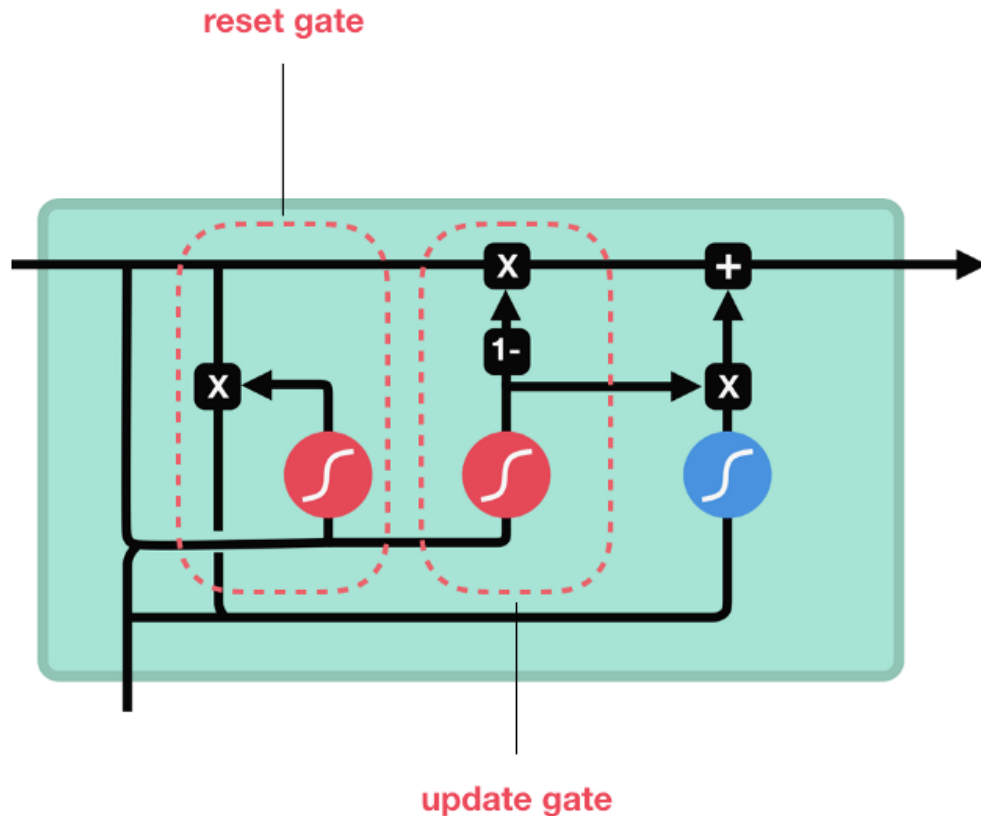
Bi-directional recurrent model



Bi-LSTM



Gated recurrent units



GRU's got rid of the cell state and used the hidden state to transfer information. It also only has two gates, a reset gate and update gate.

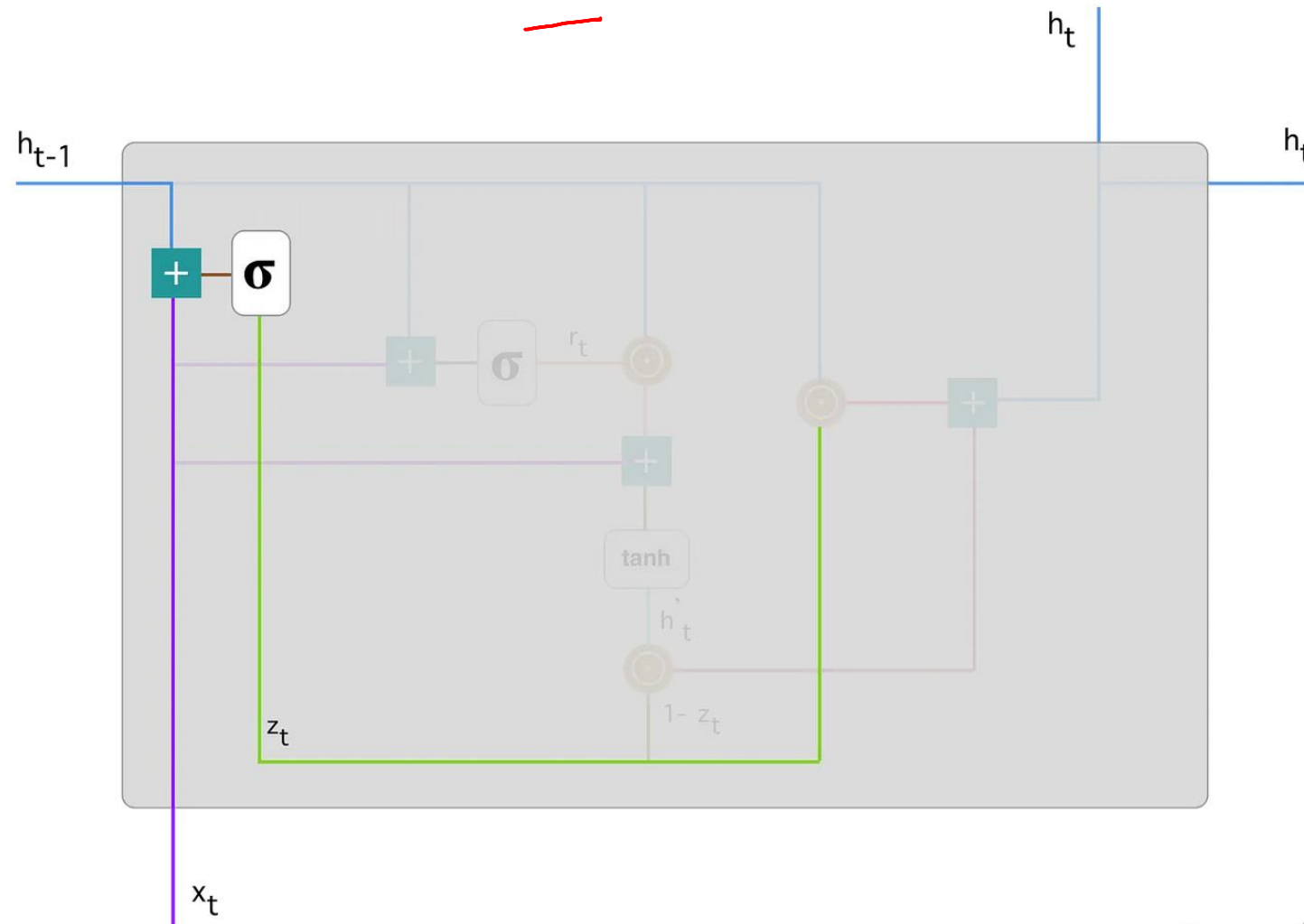
Update Gate

The update gate acts similar to the forget and input gate of an LSTM. It decides what information to throw away and what new information to add.

Reset Gate

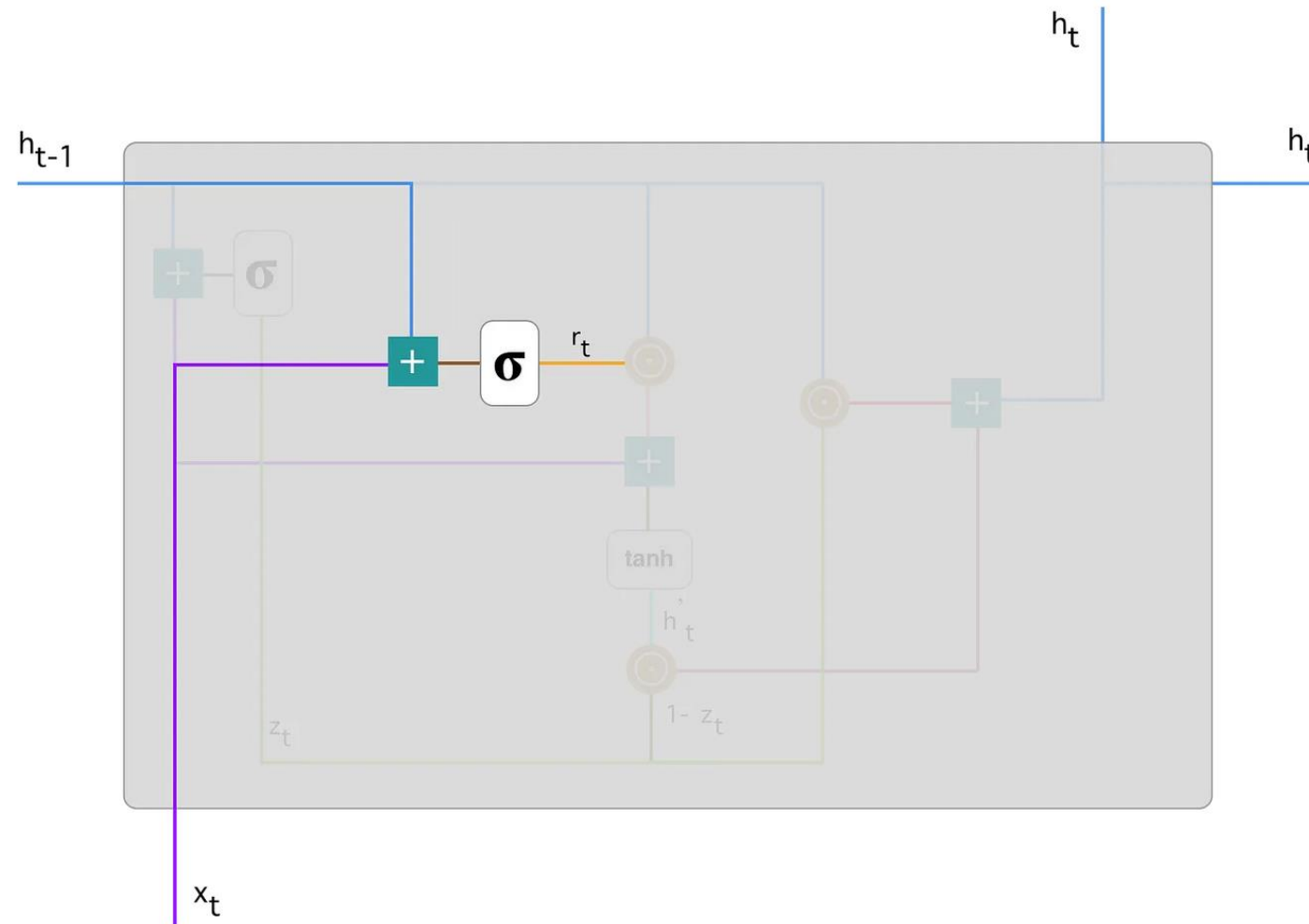
The reset gate is another gate is used to decide how much past information to forget.

Update gate



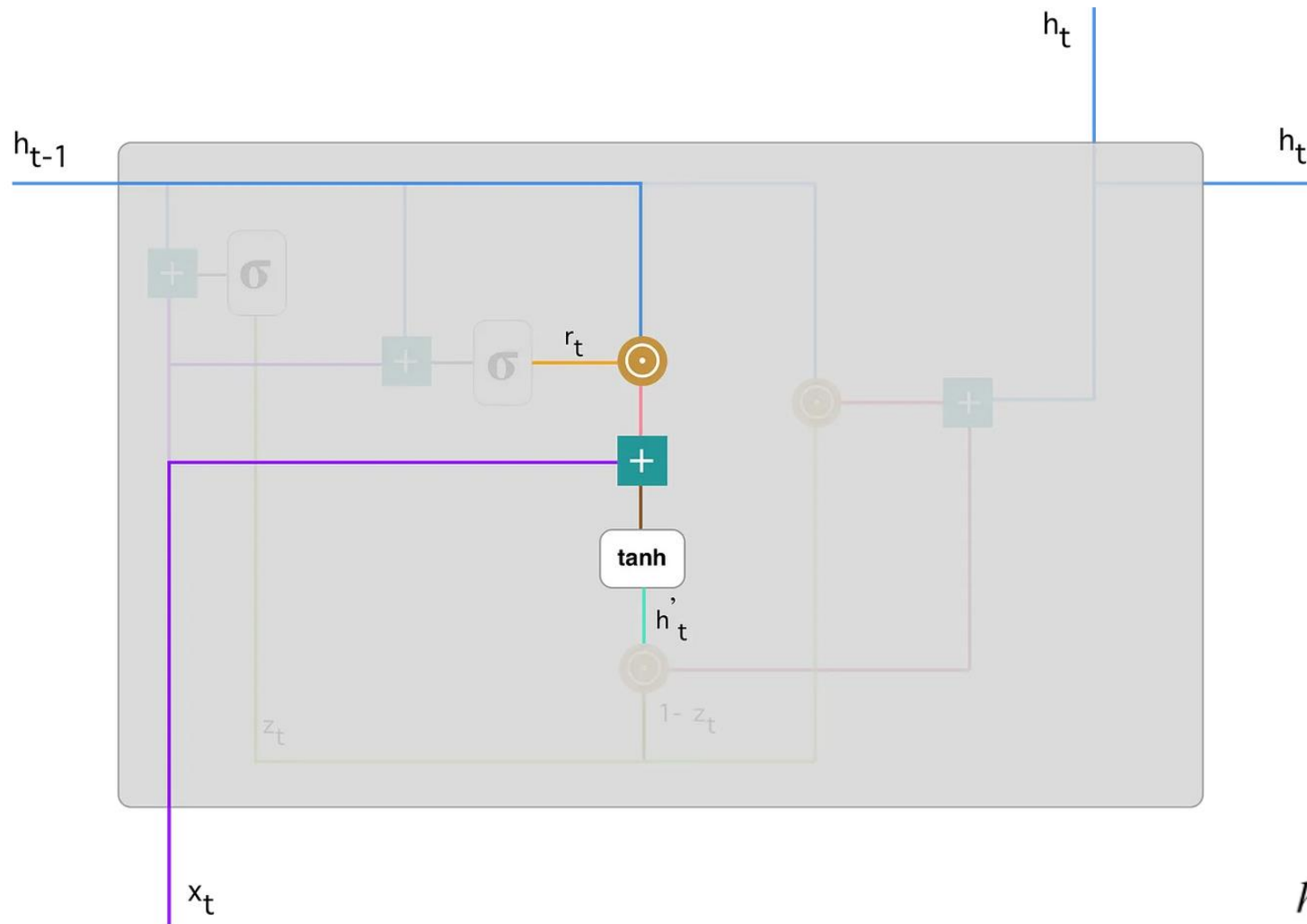
$$z_t = \sigma(W^{(z)}x_t + U^{(z)}h_{t-1})$$

Reset gate



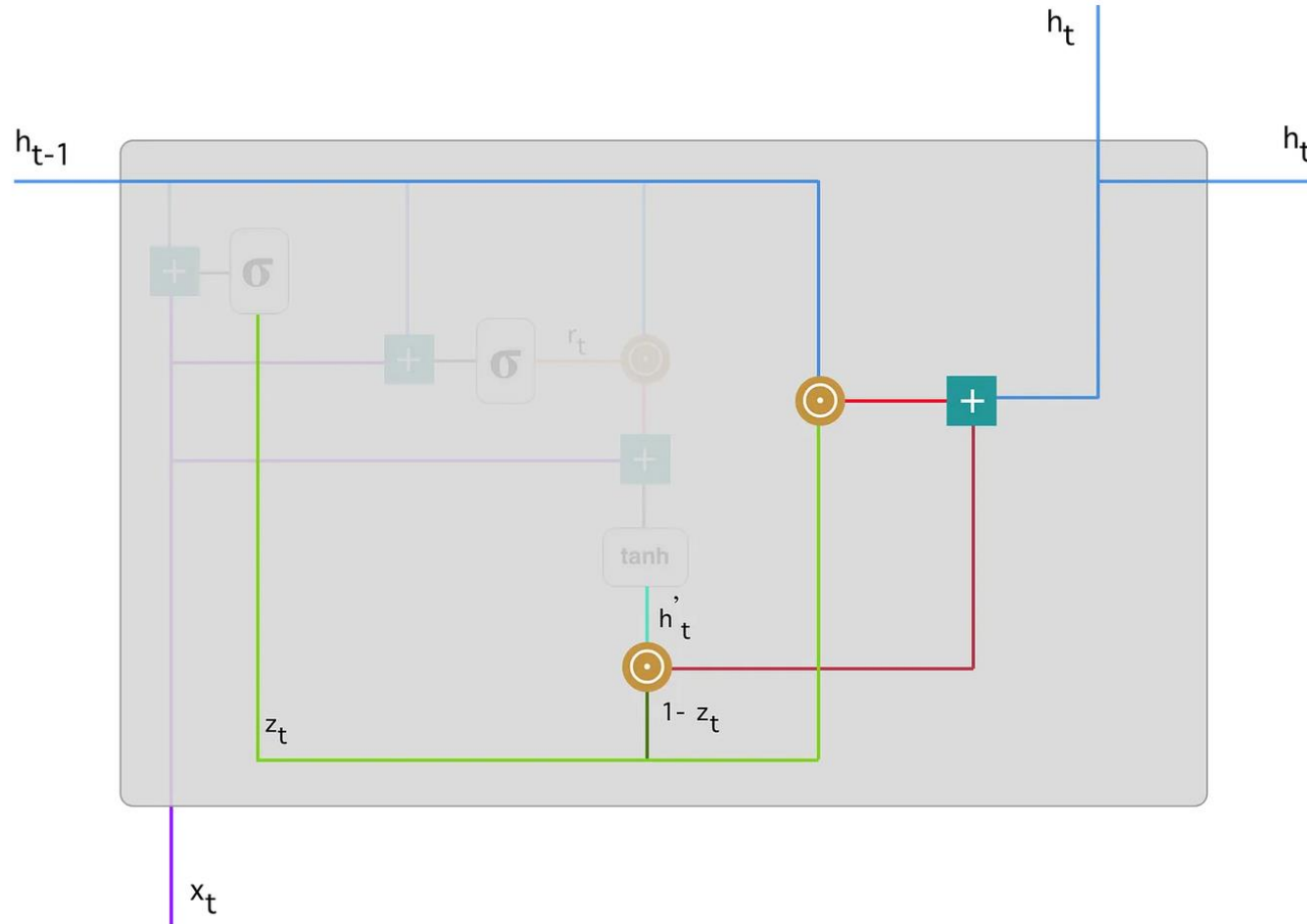
$$r_t = \sigma(W^{(r)}x_t + U^{(r)}h_{t-1})$$

Creating intermediate memory



$$h'_t = \tanh(Wx_t + r_t \odot U h_{t-1})$$

Outputting the next hidden state



$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot h'_t$$